



AgentOS-uCore 设计开发文档

基于 RISC-V uCore 的系统原生智能体运行机制

项目名称	AgentOS-uCore
队伍名称	happy-legend
目标平台	RISC-V 64 / uCore / QEMU
文档版本	决赛设计开发文档

姓名	联系方式 (QQ)
王浩泮	2091576055
康俊豪	488718235

2026 年 08 月

摘 要

智能体工作流已从一次性对话发展为持续的工具调用、文件处理和多角色协作。现有应用层框架能够安排模型和工具，却通常只在应用日志中保存角色、工作流代次、上下文因果关系和资源归属。当工具写入文件、发送消息或申请 CPU 服务时，内核仍只看见普通进程、文件和 IPC，无法据此作出有区别的判断。

本项目基于 LearningOS/uCore 实现 AgentOS-uCore。系统沿用 RISC-V uCore 的进程、虚拟内存、VFS、系统调用和调度器，并把智能体登记为内核可识别的工作负载。身份与生命周期机制将受信任映像、角色、能力位以及工作流编号和代次 `id+generation` 写入进程状态；结构化工具调用通过统一用户态接口传递工具目录、带类型信息的参数规则、执行约定和任务通道的 SQ 与 CQ；上下文（Context）、来源追溯和执行记录环保存请求、结果及数据来源；实时文件查询只面向已登记对象提供查询和订阅；智能体循环与工作流 EEVDF 则按工作流记录等待、消息、资源账户和 CPU 服务。

在集成层，AgentOS 控制台按固定步骤执行科研恢复工作流；AgentOS Nexus 则在同一次 QEMU 启动中持续运行协调、系统、研究和分析四个业务角色。Nexus 通过中继、结构化工具、消息等待和任务产物交换分派工作；客户机可同时查看身份、上下文、文件、IPC、等待和调度状态。

一次性增强实验共执行 30 次相互独立的全新 QEMU 启动，保留 33 份原始串口输出、19 份 CSV 数据表和 7,498 条逐样本记录。在 16 组独立工作流配对中，带索引核心路径的测量值均低于遍历路径，配对加速比的中位数为 **3.118 倍**。文件查询网格覆盖目录规模与命中数的 12 种组合，每组保留 15 个 AB 与 BA 的内部配对；任务通道实验保留 96 条包含 16 个操作的序列；EEVDF 实验保留 504 条精确唤醒探针，均在 0–1 个时钟滴答内获得服务。

赛题目标	完成情况	实现说明
使操作系统识别智能体身份	全部完成	受信任映像创建、角色与能力位、控制标识（ <code>control_id</code> ）、工作流生命周期和资源账户均已纳入内核对象。
形成受控工具调用路径	全部完成	工具目录、V2 与 V3 请求、参数规则解码、执行约定、重试和任务通道均通过定长用户态接口对接。
维护上下文与数据来源	全部完成	上下文镜像、路径索引、分支与回滚、来源追溯、执行记录环和工作流屏障按顺序保存请求、结果和阶段快照。
提供文件查询与实时事件	全部完成	已实现元数据目录、选择性索引、带类型信息的查询与订阅、增量缺口重新同步和 VFS 事件更新。
支持等待、协作与资源调度	全部完成	客户机场景已接入消息路由、原子等待与唤醒、心跳、资源账户和工作流 EEVDF。
验证端到端运行与性能	全部完成	控制台的恢复工作流、Nexus 的四角色协作、客户机执行约定检查和一次性增强实验均已完成验证。

表 1: AgentOS-uCore 赛题目标的完成情况

目录

摘要	ii
第一章 项目背景	1
1.1 项目背景与设计目标	1
1.1.1 智能体对操作系统的新需求	1
1.1.2 基于 uCore 的系统扩展	1
1.1.3 机制与策略的分层	1
1.1.4 相关机制与技术选择	1
1.2 设计目标与验收线索	3
第二章 系统设计	4
2.1 AgentOS-uCore 总体架构	4
2.1.1 以 uCore 为底座的产品分层	4
2.1.2 六项核心机制	5
2.1.3 一次智能体工具调用	5
2.1.4 上层运行时	5
2.2 跨模块状态约束	6
第三章 系统实现	7
3.1 内核中的智能体身份与生命周期	7
3.1.1 从进程身份到工作流中的智能体	7
3.1.2 身份字段与角色策略	8
3.1.3 七页上下文内存	8
3.1.4 生命周期代次与并发控制	8
3.1.5 随工作流创建的资源主体	9
3.1.6 验证结果	9
3.2 结构化工具执行	9
3.2.1 工具目录与参数规则	9
3.2.2 自适应的 V2 调用	9
3.2.3 四种执行路径	9
3.2.4 工具执行期间的资源使用租约	10
3.2.5 带类型信息的任务通道	10
3.2.6 验证结果	11
3.3 上下文 (Context)、数据来源与工作流屏障	11
3.3.1 从多轮输入到结构化上下文	11
3.3.2 分支、回退与淘汰	11
3.3.3 数据来源传播	11
3.3.4 工具清单与副作用检查	11
3.3.5 执行记录环	12
3.3.6 工作流屏障	12
3.3.7 验证结果	13
3.4 智能体实时查询文件系统	13
3.4.1 智能体对文件对象的结构化需求	13
3.4.2 元数据与 inode 代次	13
3.4.3 遍历查询与选择性索引	13
3.4.4 带类型信息的实时订阅与缺口重新同步	13
3.4.5 查询性能	14
3.4.6 验证结果	14
3.5 智能体循环与工作流 EEVDF	14
3.5.1 从轮询循环到内核等待	15
3.5.2 心跳与模型请求关联号	15
3.5.3 工作流资源账户	16
3.5.4 按工作流汇总的 EEVDF	16
3.5.5 唤醒延迟与公平性	16
3.5.6 验证结果	17
3.6 AgentOS 控制台综合工作流	17
3.6.1 科研恢复场景	17
3.6.2 遍历与索引路径的配对实验	17
3.6.3 核心查询与完整流程窗口	18
3.6.4 系统结果	18
3.7 AgentOS Nexus 多智能体协作平台	18
3.7.1 长驻会话与四角色协作	18
3.7.2 一次交互回合	19
3.7.3 N1 子任务协议	20
3.7.4 结果文件传递	20
3.7.5 按角色显示工具	20

3.7.6	调用级审批与取消	21
3.7.7	内核观测信息	21
3.7.8	运行验证	21
3.8	关键实现	21
3.8.1	启动、身份与 workflow 代次	21
3.8.2	结构化工具调用与执行约定	25
3.8.3	上下文、回滚边界与数据来源	33
3.8.4	元数据查询与实时查询事件	37
3.8.5	任务通道与等待路径	43
3.8.6	资源使用租约与 workflow EEVDF	45
3.8.7	客户机运行时与结果文件发布	53
第四章	项目测试	58
4.1	功能与性能测试	58
4.1.1	测试组织	58
4.1.2	一次性增强实验	58
4.1.3	任务通道传输延迟	59
4.1.4	内核 I/O 与工作量	59
4.1.5	可重复性	60
4.2	测试路径与结果	60
4.2.1	客户机原生工作负载与一次性测量	60
4.2.2	工具、执行约定与任务通道测试	62
4.2.3	Nexus 会话与宿主机中继程序	66
4.2.4	现有测试范围	68
第五章	总结与展望	69
5.1	阶段性结论	69
5.2	当前技术限制	69
5.3	后续工作	69
第六章	参考说明	70
6.1	uCore 与系统基础	70
6.2	协议、工具与运行时	70
6.3	测量资料	70
6.4	文献使用	70
第七章	参考文献	71

1. 项目背景

智能体的“计划-调用-等待-修正”循环会同时经过进程、文件、系统调用和调度器。若角色、 workflow 标识和来源关系只保存在应用层，内核看到的便只能是无差别的线程和文件描述符，因而无法在创建、副作用提交、资源占用或唤醒服务等时点作出一致判断。本章依次讨论 AgentOS-uCore 所要解决的问题、uCore 的扩展边界和设计目标。

1.1. 项目背景与设计目标

1.1.1. 智能体对操作系统的新需求

大模型工具调用已从单次问答发展为持续运行的智能体循环。典型工作流会反复执行“读取上下文、选择工具、等待结果、修正计划”，并将任务拆分后委派给多个专职智能体。这类负载具有三个突出特点：生命周期较长，工具副作用丰富，协作过程中会产生大量中间状态。

传统进程抽象记录 PID、内存、文件和 CPU 时间，却无法直接表达智能体的角色、工作流代次、上下文因果关系、工具模式和数据来源。若这些信息只保存在提示词、JSON 格式文本和应用日志中，内核便难以在文件、IPC 与资源操作实际发生前进行统一检查。单一角色还可以通过创建更多线程扩大调度份额，进而影响其他工作流获得服务的时间。

我们将 AgentOS-uCore 的目标概括为：**使操作系统能够识别智能体工作负载，并在文件、消息和资源等副作用发生时提供一致的身份、资源、因果与调度机制。**

1.1.2. 基于 uCore 的系统扩展

项目建立在 LearningOS/uCore 教学内核之上 [1]。uCore 提供 RISC-V 启动流程、进程、虚拟内存、VFS、异常处理、系统调用和基础调度机制。我们保留这些主干，只在现有 PCB、VFS 对象、调度队列和系统调用路径中补充智能体状态。所有新用户态接口均使用定长结构，并明确写入版本号和结构大小，再由内核和用户态共同检查布局。普通 uCore 进程仍沿用原有路径；智能体进程则通过受控的创建和 exec 路径取得内核身份。

文件元数据采用当前启动周期的内存附属记录，并与实际的 dev+inum+incarnation 绑定。这样既不改变原有 uCore 磁盘格式，也能分别维护索引、订阅和增量缺口后的重新同步。

1.1.3. 机制与策略的分层

文件、消息、资源和调度等必须由内核裁定的动作留在内核；目标拆分、历史整理和工具选择留在客户机用户态；传输加密、服务接口密钥和模型服务协议则留在宿主机。这样，无论使用固定策略、严格回放还是实际模型服务，客户机都经过同一条内核调用路径。

1.1.4. 相关机制与技术选择

资源记账借鉴 Linux 的 cpuacct 与 rstat 分层汇总思路 [2, 3]，执行记录借鉴 BPF 环形缓冲区的有序提交方式 [4]，调度采用 EEVDF 的滞后量与虚拟截止时间概念 [5]。任务通道采用

系统对象	uCore 基础	新增机制
进程与线程	PID、内存、文件表、运行状态	智能体身份、角色与能力位、控制标识、 workflow 生命周期
虚拟内存	页表、用户页与内核页	六页只读的上下文镜像与一页用户缓存
文件系统	inode、目录、读写与 fsync	以本次启动为范围的元数据、选择性索引、带类型信息的实时查询
系统调用	参数拷贝、执行与返回	结构化工具目录、带类型信息的 V2 与 V3、任务通道的 SQ 与 CQ
进程间通信	管道、信号与基本等待	消息路由、订阅与等待、心跳、模型请求关联号
调度与资源	线程运行队列	workflow 调度实体、资源账户、EEVDF 的滞后量与虚拟截止时间

表 1.1: uCore 基础与 AgentOS-uCore 的扩展

层次	主要对象	职责
宿主机	命令行、观察程序、模型服务适配器、QEMU 串口	负责交互、传输、加密和消息格式转换
客户机用户态	协调智能体、专职智能体、N1、任务产物存储	负责拆分目标、维护任务状态、整理任务产物和回传结果
AgentOS 内核	身份、上下文、工具、VFS、IPC、资源、调度器	负责身份检查、权限判断、数据来源记录、等待、资源记账与调度

表 1.2: AgentOS-uCore 的三级职责划分

io_uring 的 SQ 与 CQ 通信形式 [6]，文件元数据和选择性索引参考 Haiku BFS[7]。智能体工具循环中“模型选择、应用执行、结果回传”的职责划分借鉴现有工具调用实践 [8, 9]。

在 uCore 中，身份、上下文、VFS 访问范围、资源账户和 EEVDF 都记录同一个 id+generation。因此，文件查询、工具调用和调度器在判断某个智能体归属时会得到同一答案。

1.2. 设计目标与验收线索

项目把智能体的系统行为拆成四类可检查的对象：身份与生命周期说明是谁发起操作；工具、文件和 IPC 的结构化请求说明要做什么；上下文、来源追溯和执行记录说明操作如何发生；资源账户和工作流调度器说明 CPU 服务如何分配。每类对象都有对应的客户机程序和宿主机验证路径。性能实验只比较工作负载和结果指纹相同的两条路径。

问题	内核增量	对应验证
角色与工作流不能被系统识别	受信任映像、智能体身份、控制标识、生命周期代次	create、fork、exec、exit 与陈旧代次场景
工具调用参数与副作用分离	工具目录、V2 与 V3、参数规则、执行约定、任务通道的 SQ 与 CQ	ABI、参数规则、重试、取消和执行约定场景
文件与消息缺少因果关系	上下文路径、来源追溯、元数据、执行记录环	快照与分支、查询与订阅、工作流屏障和审计场景
等待与资源以线程为单位碎片化	等待与唤醒、心跳、资源账户、工作流 EEVDF	竞争、唤醒、并发度与公平性测量

表 1.3: 从智能体负载问题到 AgentOS-uCore 内核机制

2. 系统设计

AgentOS-uCore 沿用 uCore 的进程、页表、VFS、异常处理、系统调用和调度主干，不以独立运行时取代内核对象。一个智能体发起工具调用时，身份检查、上下文记录、文件访问、消息投递、资源记账和调度选择都读取进程中保存的同一组工作流编号和代次。图示、数据结构和状态机明确规定这些模块何时读取字段、何时更新状态，避免同一请求在不同位置被认作不同工作流。

2.1. AgentOS-uCore 总体架构

2.1.1. 以 uCore 为底座的产品分层

AgentOS-uCore 由直接运行在 uCore 内核中的智能体功能模块构成。图2.1自下而上展示系统结构：uCore 提供基础系统对象，AgentOS 在这些对象上增加六类机制；控制台和 Nexus 通过统一用户态接口分别完成单智能体和多智能体工作流。

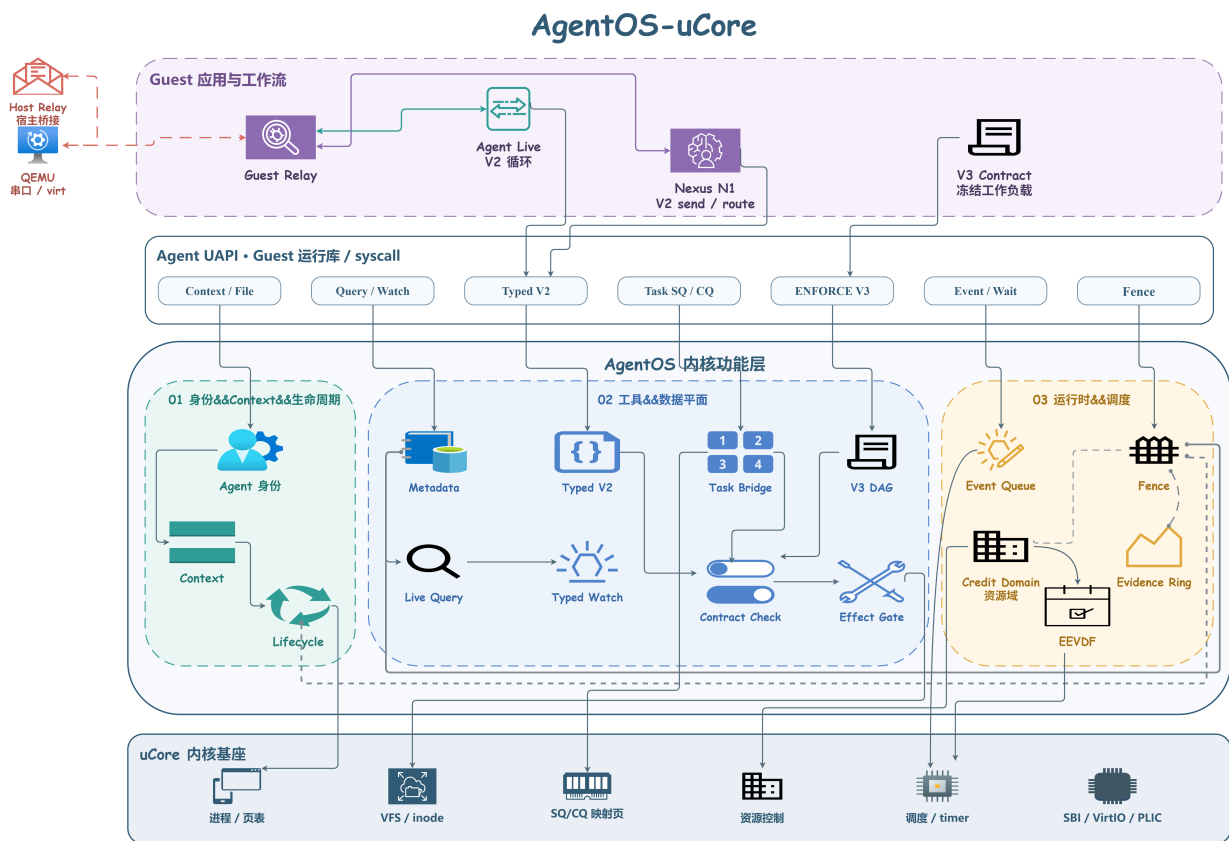


图 2.1: AgentOS-uCore 总体架构

底层 uCore 主干继续承担进程、页表、VFS、异常处理、系统调用和调度器的原有职责。AgentOS 在此基础上把相关机制分为三组：

1. **身份、上下文和资源**：身份与生命周期登记智能体；上下文保存每次工具交互；资源账户和 EEVDF 分别记录资源使用并分配 CPU 服务。

2. **工具、文件和消息**：带类型信息的工具调用检查参数规则和能力位；VFS 元数据与实时查询处理文件对象；消息路由传递跨智能体消息。
3. **来源记录和工作流结束**：上下文路径记录因果关系；来源追溯随文件、工具和 IPC 传播；执行记录环与工作流屏障分别保存阶段记录和结束时的快照。

2.1.2. 六项核心机制

模块	关键对象	主要功能
身份与生命周期	身份记录、控制标识、生命周期	创建受信任智能体，并绑定角色、能力位、工作流代次和 VFS 访问范围
结构化工具	工具目录、V2 与 V3、SQ 与 CQ	发现工具、检查带类型信息的参数，并执行可变调用、冻结执行约定和有界任务提交
上下文与来源追溯	上下文路径、镜像、执行记录环	连接请求、结果、因果关系、分支和数据来源
实时文件查询	元数据、索引、订阅器	为显式登记的文件提供属性、选择性查询、变更通知和增量缺口重新同步
智能体循环	事件队列、路由、心跳	提供原子订阅与等待、消息、模型请求关联、超时和取消
工作流资源与调度	资源账户、EEVDF 实体	按工作流汇总资源记账，并根据滞后量和虚拟截止时间分配 CPU 服务

表 2.1: AgentOS 的核心模块

六项机制使用同一个工作流 `id+generation`。一次工具调用会同时取得调用者身份、前一条上下文记录、参数规则、数据来源、资源账户和调度实体。工作流代次变化后，已经回收槽位对应的订阅器和句柄不能再指向新对象。

2.1.3. 一次智能体工具调用

一次完整调用从客户机读取工具目录开始。客户机按角色列出可用工具，再将模型结果编码为带类型信息的请求。请求进入内核后，AgentOS 依次拷入数据，检查版本和大小，检查身份与生命周期，解析工具并解码参数规则，再检查执行约定、数据来源、资源和副作用，最后调用 VFS、IPC 或其他 uCore 子系统。返回前，上下文路径写入一条新记录，下一轮可引用其序号。

当智能体等待模型、文件事件或其他角色时，`agent_wait` 将线程置于等待状态。事件入队与等待者安装按原子顺序执行，从而避免“发现队列为空后恰好错过唤醒”。

2.1.4. 上层运行时

控制台按固定策略依次执行身份检查、上下文记录、文件查询、IPC 和恢复操作，用于功能演示和配对测量。Nexus 使用同一组系统调用维持会话、分派任务、整理结果文件并连接模型服务。两者访问同一批 AgentOS 内核对象，因此确定性测试和自然语言交互遵循相同的系统规则。

图 2.2 从运行时序角度对比同一科研恢复任务的两种实现。普通 uCore 使用普通进程、文件和 IPC；AgentOS-uCore 则在这一路径中加入可识别的工作流标识、结构化工具、选择

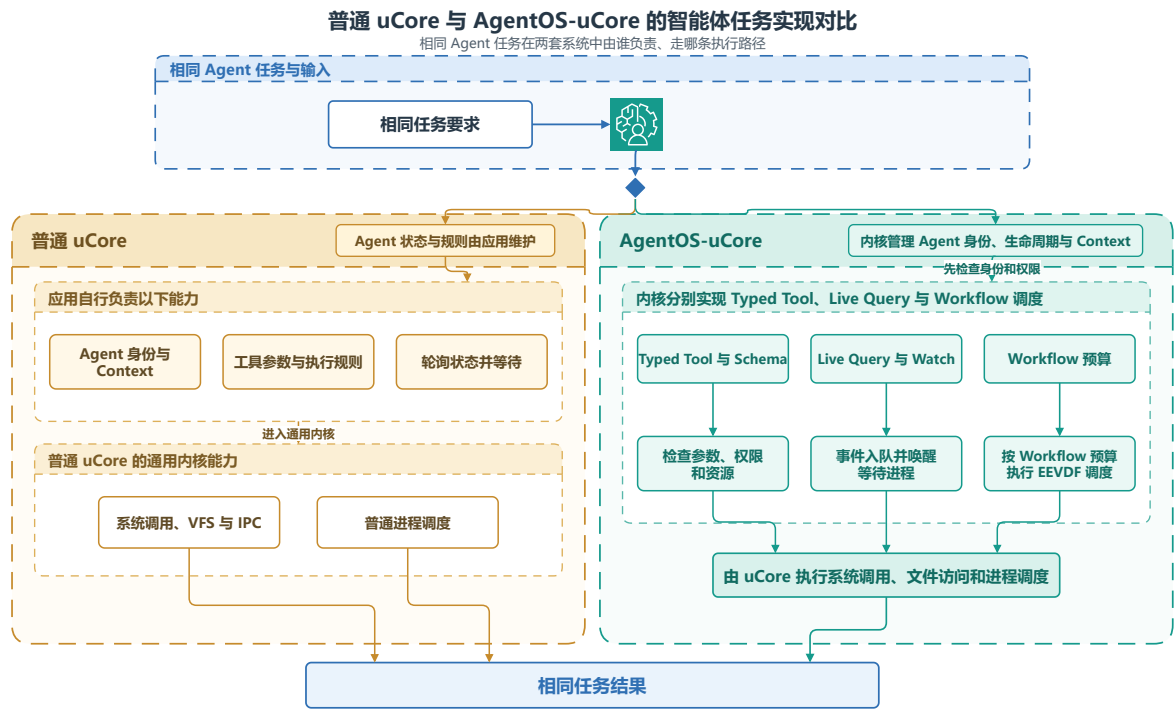


图 2.2: 普通 uCore 与 AgentOS-uCore 的智能体任务实现对比

性索引、事件等待和资源调度，使每个阶段都能够关联到明确的智能体和工作流。

2.2. 跨模块状态约束

系统将工作流标识写为可复用槽位的 id+generation。模块使用该标识前，先确认对象仍在运行、访问范围相同且代次匹配；资源、元数据和执行记录也按该标识归属。这样，旧槽位关闭、回收并再次使用后，先前的请求不会被新工作流接收。

算法 1：工作流标识的跨模块校验

输入：调用进程、目标对象、请求携带的工作流编号和代次、访问范围。

输出：可继续执行的对象引用，或明确的 STALE、DENIED 结果。

- 1. 从调用进程读取内核保存的工作流编号、代次和能力位。
- 2. 检查工作流编号和代次的结构，确认生命周期仍处于活跃状态并且代次匹配。
- 3. 比对调用者、目标对象和请求的访问范围；如不一致，则拒绝提交副作用或传递事件。
- 4. 在工具、元数据、任务通道、执行记录或调度路径中，分别执行相应的参数和资源检查。
- 5. 只有在全部检查通过后，才发布状态、将事件入队或提交资源。

3. 系统实现

本章依据当前仓库中的内核和用户态源代码。内核代码主要位于 `os/agent_*.c`、`os/workflow_*.c` 和 `os/resource_controller.c`；用户态场景位于 `user/src/`，Nexus 还使用 `user/lib/agent_nexus.c` 和 `host_tools/agentos_relayd.py`。各节先说明要解决的问题和内核采取的动作，再给出可直接定位的源代码片段。

实现部分	主要源文件	关键责任
启动与身份	<code>os/main.c</code> 、 <code>os/agent_identity.c</code> 、 <code>os/workflow_lifecycle.c</code>	初始化智能体子系统，分配角色、控制标识和工作流生命周期。
上下文与执行	<code>os/agent_context.c</code> 、 <code>os/agent_core.c</code> 、 <code>os/agent_execution_contract.c</code>	写入上下文，解析 V2 与 V3 请求，并按执行约定检查副作用。
任务与来源	<code>os/agent_task_channel.c</code> 、 <code>os/agent_provenance.c</code> 、 <code>os/agent_evidence_ring.c</code>	管理任务通道的 SQ 与 CQ、来源标签和工作流执行记录的阶段快照。
文件与事件	<code>os/agent_metadata_*.c</code> 、 <code>os/agent_live_query_events.c</code> 、 <code>os/agent_ipc.c</code>	维护元数据目录、查询与订阅、消息和等待。
资源与调度	<code>os/resource_controller.c</code> 、 <code>os/workflow_credit_domain.c</code> 、 <code>os/workflow_scheduler.c</code>	处理资源使用租约、额度、工作流级选择和资源记账。
集成场景	<code>user/src/labdemo_ucore.c</code> 、 <code>user/src/agent_nexus_ucore.c</code>	驱动恢复流程和四角色常驻协作。

表 3.1: AgentOS-uCore 的实现源代码分工

3.1. 内核中的智能体身份与生命周期

3.1.1. 从进程身份到工作流中的智能体

AgentOS 区分普通进程、协调智能体和专职智能体。VFS、IPC、工具执行器和调度器读取同一份身份记录，因此同一个进程在各处拥有相同的角色和能力位。内核将智能体身份写入 PCB，再根据受信映像清单和父角色授权完成创建。用户态可以读取身份信息，权限判断始终以内核副本为准。

创建受控工作流时，内核先分配生命周期，再写入身份、角色、能力位和控制标识，随后建立上下文页、VFS 访问范围和资源账户。任一步骤失败，内核都会回收已经分配的对象。创建完成后，各子系统都通过同一组工作流编号和代次找到该智能体所属的工作流。

`agent_worker_create()` 不会新建 workflow 根生命周期。它只在已有生命周期中创建一个携带受控待执行映像的子进程；调用者只要在同一 workflow 内具有 `ORCHESTRATE` 即可调用，不要求它必须是控制者。子进程仍须沿通常的 `exec()` 路径装入映像，所以该调用返回时，子进程尚未在新映像中运行。`agent_scope_delegate_fd()` 只接受 `FD_INHERIT_DELEGATE`，当前实现用于管道。该接口不会立即把文件描述符放入某个访问范围，而是在调用线程上登记一次性凭据；下一次创建受控子进程时，内核制作创建快照才会使用该凭据。子进程通过 `filedup()` 获得文件描述符副本，父进程继续保留原描述符，随后内核清除这张凭据。

3.1.2. 身份字段与角色策略

字段	含义	使用模块
<code>agent_id</code>	当前启动周期内的智能体编号	工具、上下文、观测
<code>control_id</code>	内核分配的控制身份	控制者绑定、IPC 路由、句柄校验
角色与能力位	角色及其具体权限集	工具、文件、消息与任务产物操作
workflow	workflow 标识和可复用槽位的代次	生命周期、订阅、任务通道、资源、调度
<code>id+generation</code>	workflow 的文件访问范围	元数据、查询、任务产物
VFS 访问范围	workflow 资源归属	进程、线程、文件、块、页与缓冲区
执行与存储账户		

表 3.2: 智能体身份的核心字段

内核的角色策略将每个角色映射为能力位掩码和调度权重。哨兵角色主要读取进程、元数据和调度状态；调查角色只能读取受限内容；报告角色可以创建报告结果文件；协调角色可以分派任务并创建角色。子智能体继承同一 workflow 的访问范围，其能力位同时受父角色授权和映像清单限制。

3.1.3. 七页上下文内存

每个智能体在固定 ABI 地址拥有七页上下文内存。前六页由内核维护，并以只读方式映射到客户机，依次提供身份头、最近返回、上下文记录、路径索引和稳定摘要。第七页是用户缓存，客户机用它保存高频策略数据。身份、能力位、访问范围和因果序列均来自前六页的内核状态。

上下文最多保存 128 条记录。每条记录包含序号、请求、工具、状态、服务起始时钟、原因序号、跨度、分支代次、控制标识和记录摘要。内核按照发布序号更新镜像；客户机也可以通过查询和快照接口读取同一结构。

3.1.4. 生命周期代次与并发控制

workflow 槽位可在回收后复用，内核每次重新分配时都会推进代次。订阅器、资源账户、VFS 附属记录、任务句柄和任务产物都同时保存标识和代次。

生命周期使用普通操作、退出操作和 workflow 屏障三类并发控制。普通系统调用增加普通操作计数；`exit` 或拆除过程增加退出操作计数；workflow 屏障在两类操作全部完成后生成阶段快照。当控制者已经关闭、成员数、活跃普通操作数和退出操作数都归零，并且屏

障已经释放时，回收程序会清除该生命周期的上下文、订阅器和执行记录环。普通 VFS 访问范围会被完整回收；对于 `preserve_on_retire` 持久输出访问范围，只要存储账户进入 DRAINING 或 FREE，内核即可回收生命周期代次，同时保留登记项、文件和存储额度。

3.1.5. 随 workflow 创建的资源主体

workflow 创建时，内核同时分配执行账户和存储账户。workflow 中的进程、线程、文件对象、文件系统块、inode、智能体状态页和物理页都记入同一账户。这样，资源使用租约和 workflow EEVDF 都按 workflow 计算，子进程和多线程也始终记在原 workflow 名下。

3.1.6. 验证结果

`agenttrust_ucore` 覆盖受信 `exec`、角色授权、能力位收缩以及 `fork`、`exec`、`exit` 与普通进程共存；`agentscope_ucore` 覆盖访问范围与代次复用；`agentfinal_ucore` 覆盖上下文固定映射、快照、分支、回滚和容量淘汰。所有场景都在 RV64 uCore 客户机中通过真实系统调用运行。

3.2. 结构化工具执行

3.2.1. 工具目录与参数规则

智能体既要发现可用能力，也必须在副作用发生前通过调用校验。AgentOS 内核维护 25 项工具目录。每个描述项都写明版本号、结构大小、工具编号、名称、参数规则、标志和副作用类别；启动时，内核检查工具编号和名称是否唯一，并检查参数规则是否完整。

`sys_tool_list` 向客户机返回定长描述项。客户机再按角色和能力位列出模型可见的工具，并补充用途、参数和结果说明。等待、上下文和模型请求等运行时接口只供客户机使用，模型只能请求当前角色允许的产品动作。

3.2.2. 自适应的 V2 调用

V2 请求明确携带版本号、请求大小、工具名称、工具编号、参数数量和带类型信息的参数数组。当前标量类型为 `uint64` 与字符串，适合 uCore 的紧凑参数解析。内核一次性拷入参数，依次检查参数名、类型、数量和长度，再进入共享执行器。

V2 适合根据上一轮结果改变下一步操作的智能体循环。例如，协调智能体可以先查询失败阶段，再根据候选数决定读取摘要还是创建研究子任务。每次调用都会独立检查身份、能力位、访问范围、参数规则、数据来源和资源状态。

3.2.3. 四种执行路径

紧凑批处理用于同步执行短操作序列。V3 则在工作流生命周期内冻结一份带版本的执行约定，其中最多包含 24 个按拓扑顺序排列的节点。每个节点关联工具、参数规则摘要、所需能力位、输入和输出任务产物类型、截止时间、尝试次数、来源与副作用清单，以及执行和存储额度。

V3 请求同时绑定执行约定代次、节点、尝试次数、来源节点、前驱上下文序号和 32 字节指纹。无论成功还是失败，完成结果都会写入固定完成槽；合法重试直接读取原结果，

路径	输入形式	主要用途	运行语义
紧凑批处理	最多 64 个 agent_op	固定顺序和上下文压力场景	在一次系统调用内顺序执行，逐项返回状态
V2	带类型信息的键值参数	依据上一轮结果选择工具	每次都检查参数规则、能力位和访问范围
ENFORCE V3	V2 前缀与执行约定封套	预先定义的高保障 DAG	冻结节点、前驱、尝试次数、截止时间和副作用清单
任务通道的 SQ 与 CQ	16 槽 SQ 与 16 槽 CQ	有界提交、完成和背压	仅主线程建立通道，绑定提交线程并校验代次；每个已接受任务恰有一条完成 CQE，取消不会额外产生 CQE

表 3.3: 四种工具执行路径

避免再次产生副作用。

3.2.4. 工具执行期间的资源使用租约

高风险工具可以在执行前从 workflow 账户预留执行和存储额度。资源使用租约依次经过 ADMITTED->ACTIVE->DEACTIVATED->SETTLED。分配器在发布对象前取得一次性编号声明，发布成功的对象继续占用同一份已用额度，析构时再结算一次。这样，多个对象在一次工具调用中同时达到资源峰值时，账户仍能准确记录用量。

3.2.5. 带类型信息的任务通道

任务通道只能由智能体主线程建立，并映射 16 槽 SQ 和 16 槽 CQ，两者各占一个客户机映射页；复制的请求和完成记录、权威资源状态则分别保存在两页内核私有页中。SQE 与 CQE 均为 128 字节。进入通道和资源绑定会记录提交线程标识及身份代次。内核先拷入完整 SQE，再检查进入接口的 ABI、递增请求编号、执行约定、节点、尝试次数、参数规则、截止时间、取消链接和带类型信息的句柄。提交者、所有者或生命周期不匹配导致 STALE 时，输出只保留 ABI 版本、结构大小和 status，其余字段全部置零；控制标识代次不匹配时，进入接口返回当前通道状态。

内核私有的头尾指针才是权威状态。CQ_FULL 只表示 CQ 暂无可写槽位，是背压，不表示已接受任务已经完成；对应任务仍待发布，直到 CQ 出现可用槽位。SQE 的环或槽位代次不一致，或者发生其他协议错误，都会使通道进入需要重新同步的状态；提交者必须以 AGENT_TASK_CHANNEL_ENTER_F_RESYNC 和空 sq_tail 显式复位，随后才能继续提交。每个已接受任务在成功、拒绝、失败、取消或超时后发布一条完成 CQE。CANCEL 命令使用独立请求编号，只引用目标任务，不会另行生成 CQE；同步策略拒绝仍会消费取消编号，并由进入接口返回 DENIED，目标任务继续保持原运行状态。目标任务已经确认或不存在时，同样会消费取消编号，进入接口返回当前代次和水位线。当前服务提供者接受空输入并返回 NONE，借此验证通道、截止时间、取消和完成语义。

3.2.6. 验证结果

`agenttoolabi_ucore` 覆盖目录发现、名称与编号一致性、未知参数、类型错误和返回缓冲区；`agentcontract_ucore` 覆盖 DAG、截止时间、重试、取消、来源上下文与来源检查；`agenttask_ucore` 覆盖 SQ/CQ 已满、重新同步、取消和完成记录保留。一次性数据集还保存了 96 条每条 16 个操作的序列，第 4 章集中分析其分布。

3.3. 上下文 (Context)、数据来源与 workflow 屏障

3.3.1. 从多轮输入到结构化上下文

智能体的下一步决策往往依赖上一次工具结果。每次工具调用即将执行可能产生副作用的操作时，AgentOS 都会建立一条上下文记录，并由附属记录保存归一化的 `agent_op`、结果和归因信息。附属记录不会完整保留 V2 参数、V3 绑定或任务通道 SQE，而是保存请求、结果、因果关系和数据来源，供后续查询。

记录序号只在同一次清空周期内递增。记录包含请求、工具、状态、服务起始时钟、原因序号、跨度、分支代次、控制标识、短载荷或结果以及记录哈希；执行清空后，序号从 1 重新开始。`path_parent_sequence` 指向当前活动头记录，后继索引用于查询分支。

上下文追加依次经过准备、填充、发布和提交；`detail` 会在记录标为奇数前写入。镜像头发布记录数、头记录、最早记录、最新记录、可见头记录、丢弃数和回滚数。客户机读取记录后重新核验序号和哈希，并再次确认摘要未变化，便可得到一致快照。

3.3.2. 分支、回退与淘汰

上下文路径最多保留 128 条记录。容量用尽后，内核按序号淘汰最早记录，并更新最早记录、最新记录和丢弃计数。活动路径只引用仍然保留的前驱记录。

回滚将 `visible_head` 移到仍然存在的上下文序号，并为后续调用分配新的分支代次。已经产生的文件、IPC 和模型服务副作用保持不变。需要幂等的副作用由 V3 尝试缓存和具体工具协议处理。因此，上下文分支只改变后续决策看到的路径；手工备注、回滚和清空都不会预留或提交执行记录。

3.3.3. 数据来源传播

AgentOS 使用六类可组合的数据来源标签，标签随上下文、文件、工具返回、IPC 和任务产物传播。

文件、工具、邮箱和任务资源按保守的按位或规则传播来源标签。生成摘要、重新计算哈希和在智能体之间整理任务产物时会增加新来源，同时保留既有标签。因此，文件数据可以供模型查看，但不会自动变成控制授权。

3.3.4. 工具清单与副作用检查

每个内核工具都有安全清单，其中列出可接受的输入来源标签、输出新增标签、所需能力位，以及文件、元数据、IPC、进程、权限、任务产物和订阅的副作用掩码。启用 ENFORCE V3 后，生命周期代次、执行约定中冻结的边、参数规则、能力位、工具清单和数据来源必须同时匹配。失败请求会在工具产生副作用前返回结构化状态。直接调用或 V3 调用被拒

标签	含义	典型来源
KERNEL_FACT	内核生成的结构化事实	PID、资源、调度器、能力位
TRUSTED_USER_CONTR OL	经产品界面提交的用户控制信息	新交互轮次、参数绑定 审批
AGENT_DERIVED	智能体根据已有输入推导的数据	摘要、计划、报告
UNTRUSTED_FILE_DAT A	来自文件的内容或元数据	查询、读取、摘要哈希
UNTRUSTED_TOOL_OUT PUT	工具执行返回	带类型信息的 V2 或 V3 结果
CROSS_AGENT_DATA	经 IPC 或任务产物在角色间传递	任务结果、协调者整理的 任务产物

表 3.4: 上下文的数据来源标签

限时通常记录来源上下文、工具和原因；但是关键记录的预留或提交失败时，调用会返回 NO_SPACE 或 RETRY，因此不能认为每次拒绝都一定有执行记录票号。

3.3.5. 执行记录环

每个活动 workflow 按需分配四个智能体状态页：三页普通记录环，共 48 个 256 字节槽位；一页关键记录环，共 16 个槽位。直接调用或 V3 调用的标准完成或拒绝路径会预留递增的执行记录票号，填充后再提交；手工备注、回滚和清空不使用这套预留和提交规则。当较早编号仍在准备时，读取者会停在当前位置，从而保持全局发布顺序。

普通成功的上下文记录写入普通记录环，获准副作用和成功建立记录的拒绝写入关键记录环。拒绝记录无法预留或提交时，直接调用或 V3 调用返回 NO_SPACE 或 RETRY，而不是伪造关键记录编号。复用环形空间前，已提交分段会汇入内部根记录。该结构为 workflow 屏障提供按执行记录票号排序的执行记录。

3.3.6. workflow 屏障

控制者通过 agent_run(count=0, AGENT_RUN_F_FENCE) 发起 workflow 屏障。内核检查协调者与控制标识的关系，使生命周期进入屏障状态，排空元数据、实时查询延迟工作和 VFS 回收，检查执行与存储账户中的待结算额度归零，最后密封执行记录环。

非空的屏障请求只携带版本号、struct_size、必须为零的 flags、必须为零的 reserved、挑战值和请求编号，**不携带**前序根记录。请求和回执可以同时为 NULL，此时内核执行匿名且不等待结果的屏障。前序根记录只由内核在阶段快照完成后写入返回回执。提供回执时，生成的 320 字节回执绑定前序根记录、挑战值、屏障序号、执行记录票号范围、事件与缺口计数、元数据代次、额度周期、精确已用额度摘要和有序分段摘要。请求编号非零且提供回执时，内核在内部状态提交后复制结果；同一个请求编号再次复制时返回缓存结果。该回执表示当前启动周期内的工作流阶段快照，其标志同时记录元数据是否易失以及覆盖状态。

3.3.7. 验证结果

`agentfinal_ucore` 连续执行 64 次工具调用，并覆盖快照、分支、回滚和窗口淘汰。数据来源场景把文件来源传入受保护的副作用，验证副作用前的拒绝与关键记录环记录。屏障场景覆盖挑战值、重试缓存、待结算额度、元数据代次和结果复制重试。

3.4. 智能体实时查询文件系统

3.4.1. 智能体对文件对象的结构化需求

科研和恢复工作流常常需要回答两个问题：当前哪些对象满足条件，哪些对象刚刚发生变化。目录遍历只能找到路径，工作流还需要项目、运行、阶段、状态、类别、依赖关系和内容摘要等属性。`AgentOS` 在 `uCore VFS` 之上增加按启动周期隔离的元数据目录，把显式登记的文件组织为可查询、可订阅的智能体对象。

3.4.2. 元数据与 `inode` 代次

拥有 `META_WRITE` 的智能体调用 `agent_file_meta_set()` 写入元数据，字段与真实 `dev+inum+incarnation` 绑定。更新通过 `update_mask` 完成，删除使用显式 `DELETE` 动作。文件创建和重命名仍沿用 `uCore VFS`，只有工作流明确关心的对象才会进入元数据目录。

对象代次用于区分 `inode` 编号复用前后的对象。`VFS` 以增量方式维护它，并随查询和订阅结果返回。查询或订阅命中记录后，内核会复核元数据目录代次、生命周期、访问范围和完整条件。`unlink`、内容大小变化和对象失效都会更新已登记的附属记录。

3.4.3. 遍历查询与选择性索引

元数据目录为 `status`、`stage` 和 `kind` 维护选择性索引。带类型信息的查询只携带条件字段、查询标志和最大命中数；访问范围由调用者身份限定，查询计划由内核选择。结果返回命中数、候选记录数、扫描记录数、全部命中数、实际返回数、`truncated`、查询计划、计划原因和文件系统代次。调用者可以用 `SCAN` 强制遍历，也可以用 `USE_INDEX` 允许内核使用索引。启用索引时，内核按固定的 `status->stage->kind` 顺序选择第一个已给定字段，不比较字段基数；索引路径仍会对候选记录检查完整条件，因此遍历与索引查询返回相同结果。最大返回数为 8，`total_hits` 仍报告全部命中数；当前 `ABI` 没有分页游标。

3.4.4. 带类型信息的实时订阅与缺口重新同步

`agent_live_watch()` 安装一份完整的 `agent_file_query`。订阅器只保存查询条件、访问范围和代次，不保存持久结果集。每次元数据变化时，内核重新计算变更前是否满足条件：对象从未命中变为命中时产生 `ENTER`，对象持续命中且记录改变时产生 `UPDATE`，对象从命中变为未命中或被删除时产生 `LEAVE`。事件以 `AGENT_EVENT_FILE_QUERY` 写入目标智能体的上下文和有界队列。

队列或待处理表无法保存全部增量时，内核会推进 `resync_generation` 并发送 `RESYNC_REQUIRED`。正常连续的文件系统代次变化仍只发布 `ENTER`、`UPDATE`、`LEAVE`，不会触发重新同步。恢复时，智能体保留旧订阅，先以相同条件安装不带 `ACK_RESYNC` 的替

换订阅，再执行有界查询。只有 `truncated=0` 的快照才能作为完整基线；随后才在旧订阅上以对应代次执行 `ACK_RESYNC` 并取消旧订阅，替换订阅覆盖整个切换时段。单次查询最多返回 8 个命中项，当前没有分页游标；快照一旦截断就不能重新建立基线或确认，调用者必须收紧查询条件，分页留待未来 ABI。重新同步不能把工作流屏障当作通用屏障。关闭工作流只会使生命周期进入关闭状态，不会排空待处理项；屏障会排空删除标记、内容增量并尝试投递标记事件。只要该工作流或访问范围中的重新同步尚未由 `ACK_RESYNC` 确认，即使标记事件已投递，屏障仍返回 `RETRY`，最终回收才会清理残留对象。

3.4.5. 查询性能

实验使用目录规模 24、64、96 与精确命中数 1、2、4、8，组成 12 个实测网格。每个网格保留 15 个 AB/BA 内部配对，并记录遍历和索引查询的延迟、候选数和检查记录数。

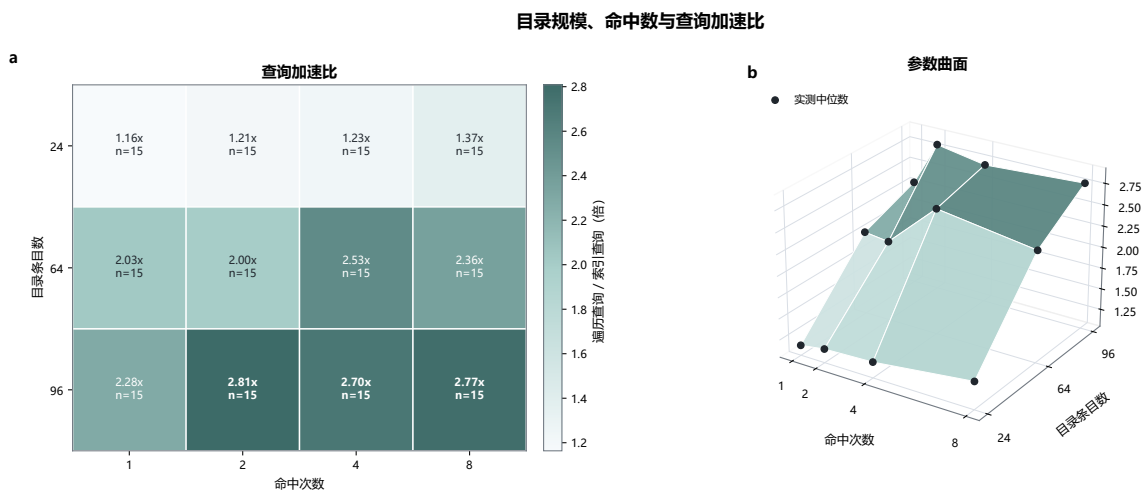


图 3.1: 目录规模、命中数与查询加速比

图3.1的 12 个格点均来自实测，中位加速比介于 1.164–2.808 倍。文件目录越大、命中项越集中，选择性索引跳过的无关记录越多。三维曲面显示这一趋势，热力图中的每个数字对应原始网格样本。

图3.2分组展示显式预热后的第一次计时遍历、后续重复遍历和就绪索引。28 条诊断记录均为 `cache=ready`，因此该图比较的是查询计划差异，而不是宿主页缓存的冷启动影响。

3.4.6. 验证结果

`agentfs_ucore` 覆盖元数据设置、更新、删除、摘要、摘要哈希、对象代次和访问范围；`agentbench_ucore` 使用同一数据集比较遍历与索引查询的结果集、候选数和延迟。实时查询场景覆盖 `ENTER`、`UPDATE`、`LEAVE`、队列缺口、`RESYNC_REQUIRED` 以及替换订阅、快照、确认的安全切换。

3.5. 智能体循环与工作流 EEVDF

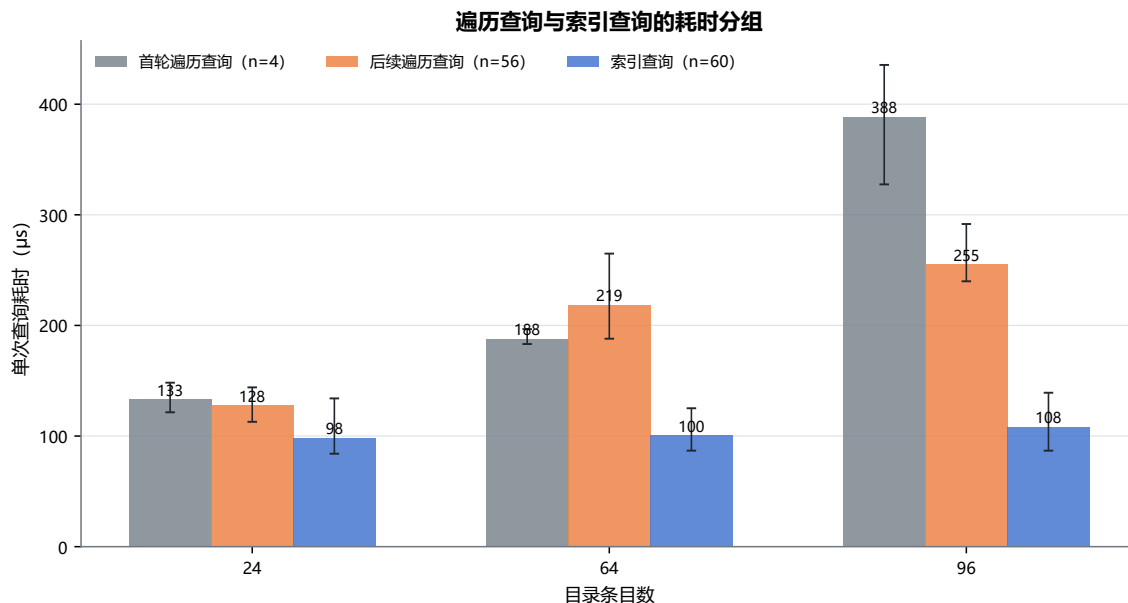


图 3.2: 遍历查询与索引查询的耗时分组

3.5.1. 从轮询循环到内核等待

智能体循环会等待用户输入、模型回复、文件变化和其他智能体的消息。持续轮询不仅频繁进入内核，还会让大量等待线程占用调度时间。AgentOS 使用有界事件队列、订阅和等待接口，让没有事件的线程真正睡眠。

每个智能体拥有容量为 16 的事件队列，其中保留一部分内核槽位，同一事件来源最多排队 4 条事件。路由表同样具有固定容量。发送 MESSAGE 时，内核检查 MESSAGE_SEND 能力位、活动 workflow、发送者、目标路由和订阅状态。公开的 agent_event 只返回发送者、目标 PID、关联号等事件字段；control_id 和数据来源标签保存在内核附属记录中。

非 TIMEOUT 的 agent_wait 会先检查已排队事件。选中事件时，内核建立保留项；没有事件时，内核以线程身份代次作为等待键安装等待者并让线程睡眠。事件入队和等待者状态使用相同锁序，确保到达事件与等待代次对应。TIMEOUT 路径在本地直接返回，不建立保留项，也不写入上下文。agent_wait_cancel() 只向等待路径注入 AGENT_EVENT_CANCELLED，它不是工具或任务通道的 CANCEL 操作；消费事件后才更新循环状态和上下文。

3.5.2. 心跳与模型请求关联号

智能体可以设置心跳间隔。定时器到期时，内核生成定时事件并唤醒智能体；客户机再根据当前状态决定检查文件、发送消息或继续等待。停止心跳时，内核同时清理绑定到代次的定时器和等待者。

LLM_REQUEST 与 LLM_RESPONSE 复用消息与等待机制。请求会登记生命周期、请求者和中继程序的 PID、控制标识、关联号和截止时间。同一请求者的关联号严格递增，并且只有成功入队后才更新。中继程序返回匹配响应时，内核原子消费待处理请求并生成 LLM_DONE。只有同一关联号仍处于活动待处理状态时才返回 DUPLICATE；复用已经完成或过期的关联号则返回 CONFLICT。待处理请求使用 120 秒内核时钟 TTL，完成历史区分已消费请求和超时请求。

3.5.3. 工作流资源账户

资源账户维护已用额度 U 、待结算额度 P 、空余额度 F 和总额 $U + P + F$ 。当前账户的可用额度先在本地图态中转移：

reserve:F→P commit:P→U cancel:P→F release:U→F

账户补充额度时会检查全局限制、资源类别和账户硬限制。系统压力、上下文切换、账户关闭和工作流屏障都会收缩 F 。工作流屏障在同一把锁内执行 trim+snapshot，要求 $P = 0$ ，并导出精确 U 、账户代次、额度周期和向量摘要。

3.5.4. 按工作流汇总的 EEVDF

传统线程级公平会让多线程工作流得到更多调度候选位。AgentOS 将同一资源账户中的线程合并为一个工作流实体。调度器使用工作流虚拟运行时间计算滞后量，满足 $vruntime \leq global_vtime$ 的实体进入可服务集合，再从中选择虚拟截止时间最早的实体。延迟类别、事件和截止时间可以缩短服务请求。一次分派完成后，内核根据实际服务周期更新工作流虚拟运行时间；睡眠实体的滞后量会向全局虚拟时间衰减。

只有一个工作流实体时，调度器使用快速路径。实体表容量为 4，其中包含一个 BOOT_SEALED 启动参与者，因此最多可同时纳入三个新建工作流。实体信息不完整或容量超出时，调度器回到原 uCore 路径，并增加回退计数。

3.5.5. 唤醒延迟与公平性

一次性实验使用 6 次全新 QEMU 启动，覆盖并发度为 1、2、3、4 的服务窗口和精确唤醒探针。504 条探针直接读取调度器跟踪中的 `ready_age`，其中 425 条为 0 个时钟滴答，79 条为 1 个时钟滴答。uCore 的时钟滴答为 10 ms；这些样本表明，在该负载下所有新建等待者都在一个时钟滴答内获得调度。

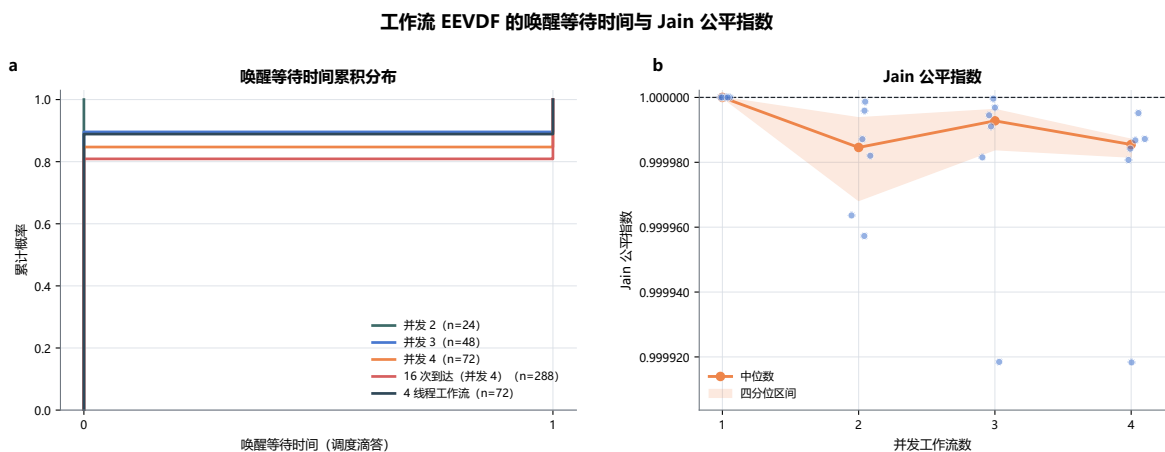


图 3.3: 工作流 EEVDF 的唤醒等待时间与调度公平性

以同一服务窗口的原始周期计算 Jain 公平性，并发度为 1、2、3、4 时的中位数分别为 1.000000、0.999985、0.999993 和 0.999985。图 3.3 中的 ECDF 保留每个唤醒探针，折线展示不同并发度下工作流服务量的均衡程度。

3.5.6. 验证结果

agentloop_ucore 覆盖原子等待、唤醒、超时、取消、心跳和消息路由；agentsched_ucore 与 agent_eevdf_ucore 覆盖快速路径、实体容量、线程放大、睡眠衰减、回退、唤醒直方图和 Jain 公平性。

3.6. AgentOS 控制台综合 workflow

3.6.1. 科研恢复场景

AgentOS 控制台用六组内核机制执行一个确定性的科研恢复 workflow。labdemo_ucore 创建协调、哨兵、调查和恢复四个角色，建立 workflow 身份、MESSAGE 路由、上下文、元数据和资源账户。场景登记 prepare、align、analyze、report、archive 五个阶段及其依赖关系，再查询失败节点、读取摘要、向其他智能体发送恢复请求，完成 align 恢复和 report 更新。

固定策略为每个阶段指定相同的业务工作量，工具结果仍由 AgentOS 内核执行和检查。因此，该场景同时覆盖身份、上下文、实时查询、IPC 等待和恢复写入，并能在不同实现路径之间进行等量比较。

3.6.2. 遍历与索引路径的配对实验

综合场景保留遍历实现和索引实现，两者功能等价。每次独立 QEMU 启动按 A/B 或 B/A 顺序运行，两侧产生相同的工作负载和结果指纹。核心窗口覆盖查询、恢复写入、fsync 和复核，查询区段单独记录纯查询时间。

规范数据集包含 16 个独立 workflow 启动，AB 与 BA 各占一半。16 个核心查询区间中，索引路径的延迟均更低；遍历和索引路径的核心中位数分别为 34.713 ms 与 13.294 ms，配对加速比中位数为 3.118 倍。

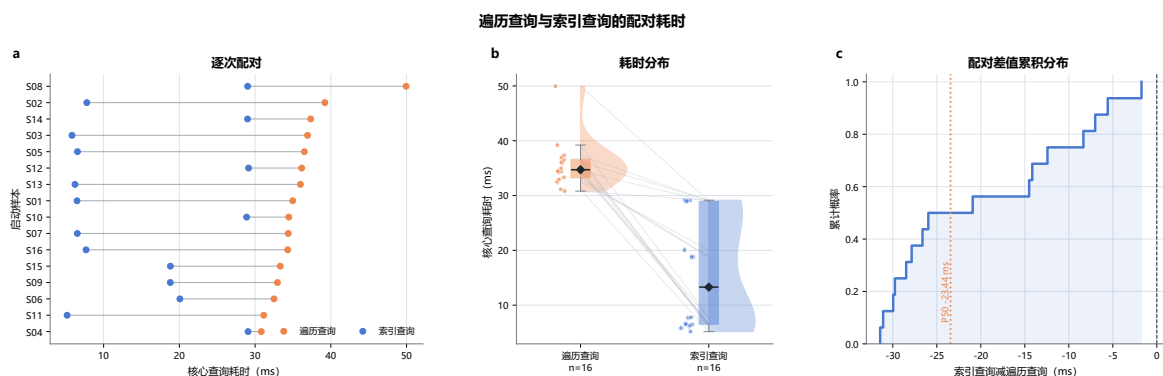


图 3.4: 遍历查询与索引查询的配对耗时

图3.4中的哑铃图逐次启动连接两条路径，右侧分布图显示各自的样本形状，ECDF 直接展示配对差值。所有配对均落在索引路径延迟更低的一侧，同时保留不同启动过程的波动。

3.6.3. 核心查询与完整流程窗口

核心窗口用于观察 AgentOS 查询路径，完整流程窗口还包含场景启动、恢复安排、fsync、复核和统计输出。在完整流程窗口中，索引路径只在 16 个配对中的 3 个取得更低延迟，indexed-traversal 的中位数为 +13.452 ms。这说明当前总时间主要受恢复写入、fsync 和复核阶段影响，索引带来的核心查询收益仍需与这些阶段一起优化。

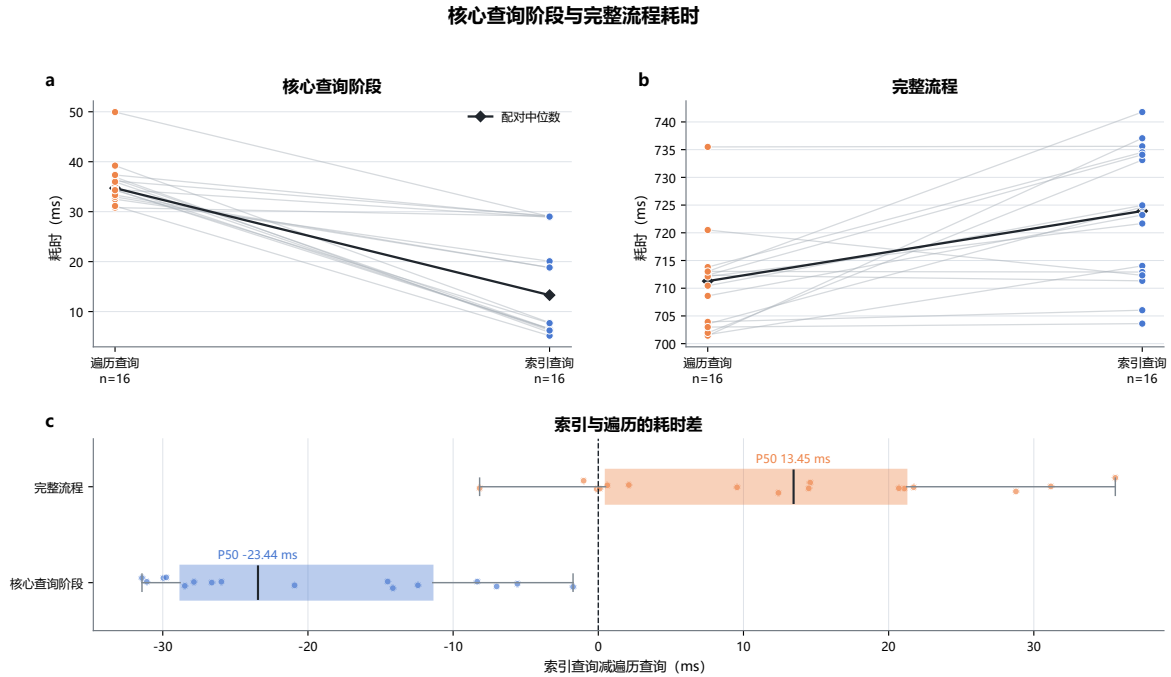


图 3.5: 核心查询阶段与完整流程耗时

AB 与 BA 两种顺序下的核心加速比中位数分别为 1.4535 倍和 5.4805 倍。顺序差异说明配对设计十分必要：每次启动都保留两条路径的原始时间，汇总时再分别展示 AB 和 BA。

3.6.4. 系统结果

综合 workflow 依次执行智能体创建、上下文记录、带类型信息的工具调用、实时查询、消息等待、恢复写入和执行记录。输出包含业务指纹、各阶段状态、检查记录数、核心与完整流程时间以及工作量计数。两条目标路径分别构建普通 uCore 与 AgentOS-uCore，并在相同的用户态约定下比较系统实现。

3.7. AgentOS Nexus 多智能体协作平台

3.7.1. 长驻会话与四角色协作

AgentOS Nexus 是构建在 AgentOS-uCore 之上的交互运行时。一次 Nexus 会话在同一 workflow 中常驻协调、系统、研究和分析四个业务智能体。每个角色都有独立的进程编号 (PID)、智能体编号、控制标识、角色、能力位和上下文。四个角色共享 workflow 生命周期、虚拟文件系统 (VFS) 访问范围、资源账户和 EEVDF 实体，资源记账与调度均按 workflow 进行。

另有一个客户机中继程序负责 QEMU 串口帧。宿主机中继程序持有本地套接字、传

产品角色	内核角色	职责	主要权限
协调智能体	协调者	维护会话、调用模型、创建子任务、核验返回	分派任务、受控工具、整理结果文件
系统智能体	哨兵	读取当前启动的进程、上下文、文件大小和调度状态	读取元数据、进程和调度状态
研究智能体	调查者	读取已登记的本地资料和历史测量结果文件	读取内容、查询和摘要哈希
分析智能体	报告者	合并系统和研究结果并生成报告	读取、写入和发布报告结果文件

表 3.5: Nexus 的四个业务角色

输加密配置、服务接口密钥和模型服务格式适配器；客户机中继程序分别转发模型请求、控制消息和运行指标帧。业务文件和结果文件保存在客户机 VFS，协调智能体根据实际工具返回决定下一步任务。

3.7.2. 一次交互回合

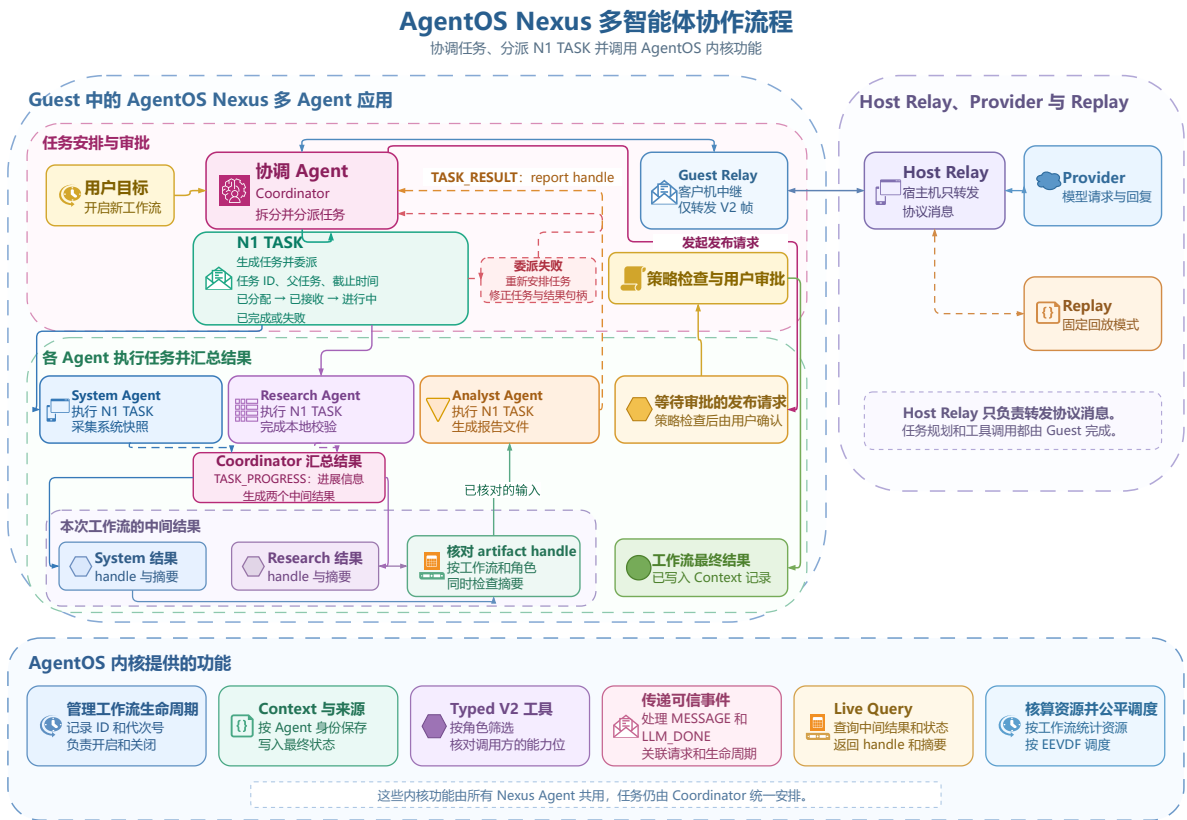


图 3.6: Nexus 多智能体协作流程

图3.6展示了从用户目标到最终结果的一次完整运行过程。

1. 命令行将新目标作为一次交互轮次提交给协调智能体。
2. 协调智能体整理有限历史和按角色筛选的工具集，发出 LLM_REQUEST 后进入 WAITIN

G_LLM。

3. 客户机中继程序将请求交给实时模型服务或固定回放模式；响应到达后，AgentOS 唤醒协调智能体。
4. 模型需要专门信息时，协调智能体使用 N1 创建系统、研究或分析子任务。
5. 子任务智能体以自身身份、上下文和能力位执行带类型信息的 V2 工具，等待期间进入 WAITING_EVENT。
6. 系统和研究智能体以 TASK_PROGRESS 进展信息回传协调智能体；协调智能体把较长结果整理为 workflow 范围内的结果文件。分析智能体自行发布报告结果文件，并以 TASK_RESULT 返回报告句柄；协调智能体取得该句柄后提交经过审批的发布请求。
7. 工具结果、结构化失败和审批结果都会返回模型，使下一轮能够继续执行或重新安排任务。

3.7.3. N1 子任务协议

N1 是客户机用户态的 44 字节规范小端封套。它以 N1: 加无填充 Base64URL 的形式，通过带类型信息的 V2 send_message 传输。每条任务包含生命周期标识、代次号、任务标识、父任务标识、发送者、接收者、截止时间、状态和有界值。状态机包含 TASK_ASSIGN、TASK_ACCEPT、TASK_PROGRESS、TASK_RESULT、TASK_FAILED 和 TASK_CANCEL。

AgentOS 内核检查 MESSAGE 层的发送者、接收者、路由、访问范围、数据来源、队列和唤醒。Nexus 运行时负责 N1 解码、状态转换校验、截止时间和单个在途任务。较长的目标、工具结果和报告通过结果文件句柄引用，因此控制消息保持紧凑。

3.7.4. 结果文件传递

结果文件清单绑定生命周期、句柄代次、类别、任务、父任务、生产者、所有者、整理者、权限掩码、载荷大小、数据来源和来源追溯。客户机用流式 SHA-256 覆盖完整载荷，读取者同时校验清单和摘要哈希。

系统和研究智能体不在任务完成时直接发布结果文件，而是通过 TASK_PROGRESS 进展信息回传协调智能体；协调智能体负责整理对应结果文件，并保留逻辑生产者和整理者。分析智能体读取两份上游结果文件后，以自身身份发布报告类别结果文件，并以 TASK_RESULT 返回报告句柄；协调智能体取得句柄后才提交经过审批的发布请求。来源标签取各输入的并集，因此报告可以同时保留文件、工具和跨智能体来源。

3.7.5. 按角色显示工具

Nexus 启动时从 AgentOS 内核读取 25 项工具目录，检查名称、编号和参数规则，再按业务角色、内核角色、能力位和副作用类别筛选。tool_search 按类别分页返回详细说明。

默认读取面包含 pid_info、ctx_stat、query_process、get_system_status、read_context、query_file、read_file_summary、read_file_digest、dependency_query 和 read_message。delegate_task、read_artifact 和 publish_report 是 Nexus 的用户态功能，协调智能体将它们组织为 N1、结果文件和带类型信息的 V2 操作。

会话保留最近 4 个完整交互轮次，每个轮次最多执行 8 次模型请求，更早内容写入结

构化摘要。每次模型请求都携带当前可见的参数规则和上一次稳定工具结果，使工具输出可以直接参与后续决策。

3.7.6. 调用级审批与取消

协调智能体取得报告句柄后，`publish_report` 的审批对象包含会话、交互轮次、请求、关联号、工具编号、规范参数摘要、一次性编号、发出时钟和过期时钟。协调智能体所在的客户机收到决定后逐字段比较；通过时，再执行精确选择器的带类型信息 `V2 artifact_update`。固定回放模式包含一次拒绝发布，报告结果文件仍然可读，会话继续返回最终结果。

`Ctrl-C` 取消当前交互轮次，常驻智能体和 `QEMU` 继续运行；`/quit` 关闭会话。关闭时依次停止观察程序的生产者、排空内核观测事件、等待观察程序退出、关闭遥测写入器；中继程序在 `EOF` 后输出 `SESSION_CLOSED`。

3.7.7. 内核观测信息

`Nexus` 观察程序通过 `agent_timeline_wait/read`、`agent_audit_query`、`agent_info` 和上下文快照生成两类视图。`kernel_audit` 显示 `MESSAGE` 入队、消费时的生命周期、身份、发送者、接收者、关联号和状态；`kernel_snapshot` 显示四个业务智能体的上下文序号、等待、唤醒间隔、调度分派、预算、虚拟运行时间和能力位掩码。

协调智能体在分配子任务和收到子任务回复时主动读取共享审计游标，观察线程使用同一把互斥锁访问该游标。左侧窗口显示交互与工具结果，右侧窗口同步显示 `AgentOS` 对应的消息、等待、唤醒和调度状态。

3.7.8. 运行验证

自动验收使用固定回放模式。每条固定响应都与客户机当次产生的规范 `MODEL_REQUEST SHA-256` 绑定，并在当前 `RV64` 镜像的真实 `QEMU` 启动中执行。基准场景包含 3 个连续交互轮次、4 个业务智能体身份、系统快照、研究句柄过期失败后的重新安排任务、分析智能体合并报告、一次审批拒绝和完整会话关闭。实时模式复用同一个客户机、`N1`、结果文件和带类型信息的工具路径，并通过宿主机模型服务适配器对接实时模型服务。

3.8. 关键实现

下列代码把设计章节中的机制逐一对应到当前代码树。代码框保留相对路径和物理行号，便于直接核对内核实现。所选片段覆盖启动、身份、调用、执行约定、上下文、文件事件、任务通道、资源和调度，说明一个智能体工作流如何创建、执行并结束。

3.8.1. 启动、身份与 workflow 代次

内核入口先初始化进程、控制台和页表等基本设施，再调用 `agentinit()`。文件系统就绪后，系统启用智能体存储和文件代次机制。随后加载的初始用户程序直接使用已建立的生命周期、元数据和资源机制，不需要在用户态临时拼接旁路状态。

代码 1: uCore 启动序列中的 AgentOS 初始化

c

```

19 void main()
20 {
21     clean_bss();
22     printf("hello world!\n");
23     proc_init();
24     console_init();
25     agentinit();
26     kinit();
27     kvm_init();
28     if (kalloc_physical_policy_init(PHYSICAL_PAGE_SYSTEM_RESERVE,
29                                     PHYSICAL_PAGE_ORDINARY_LIMIT) < 0)
30         panic("physical page policy");
31     trap_init();
32     plicinit();
33     virtio_disk_init();
34     binit();
35     fsinit();
36     timer_init();
37     agent_storage_init();
38     fs_epoch_runtime_enable();
39     load_init_app();
40     infof("start scheduler!");
41     /* 仅启动期使用的映像加载完成后, 才开放运行时输入输出准入。 */
42     bio_policy_start();
43     virtio_disk_runtime_start();
44     scheduler();

```

生命周期表用 `id+generation` 共同定位 workflow；槽位复用时，代次单调增加。创建工作流时，内核先准备记录，再一次写入创建者、访问范围、成员数和代次。因此，旧代次的订阅、任务或资源句柄会在下次检查时失效，不会误指向新 workflow。

代码 2: 工作流生命周期的创建与世代绑定

c

```

56 static struct workflow_lifecycle_record *
57 workflow_lifecycle_find_locked(struct workflow_lifecycle_key key)
58 {
59     uint slot;
60     struct workflow_lifecycle_record *record;
61
62     if (!workflow_lifecycle_key_valid(key) ||
63         key.id > WORKFLOW_LIFECYCLE_CAP)
64         return 0;
65     slot = key.id - 1;
66     record = &workflow_lifecycles[slot];
67     if (!record->used || record->generation != key.generation)
68         return 0;
69     return record;
70 }
71
72 static struct workflow_lifecycle_key
73 workflow_lifecycle_record_key(uint slot,
74                               const struct workflow_lifecycle_record *record)
75 {

```



```

76     struct workflow_lifecycle_key key = {
77         .id = slot + 1,
78         .generation = record->generation,
79     };
80
81     return key;
82 }
83
84 int workflow_lifecycle_create(uint scope_id,
85                             struct workflow_lifecycle_key *key)
86 {
87     int enabled;
88     int result = -1;
89     uint allocated_slot = WORKFLOW_LIFECYCLE_CAP;
90     uint64 next_generation = 0;
91
92     if (key == 0 || scope_id < VFS_SCOPE_FIRST_DYNAMIC ||
93         scope_id >= FS_OWNER_SCOPE_FLAG)
94         return -1;
95     *key = workflow_lifecycle_none();
96     enabled = intr_save();
97     for (uint i = 0; i < WORKFLOW_LIFECYCLE_CAP; i++)
98         if (workflow_lifecycles[i].used &&
99             workflow_lifecycles[i].scope_id == scope_id)
100             goto out;
101     for (uint i = 0; i < WORKFLOW_LIFECYCLE_CAP; i++) {
102         struct workflow_lifecycle_record *record =
103             &workflow_lifecycles[i];
104         uint64 generation;
105
106         if (record->used ||
107             workflow_lifecycle_generations[i] == ~0ULL)
108             continue;
109         generation = workflow_lifecycle_generations[i] + 1;
110         if (generation == 0 ||
111             !agent_identity_lease_lifecycle_contains(i, generation))
112             continue;
113         workflow_lifecycle_generations[i] = generation;
114         record->used = 1;
115         record->scope_id = scope_id;
116         record->generation = generation;
117         record->controller_control_id = 0;
118         record->next_context_branch = 0;
119         record->fence_sequence = 0;
120         record->members = 1;
121         record->active_operations = 0;
122         record->departing_operations = 0;
123         record->fence_gate = 0;
124         record->closing = 0;
125         *key = workflow_lifecycle_record_key(i, record);
126         allocated_slot = i;
127         next_generation = generation == ~0ULL ? 0 : generation + 1;
128         result = 0;
129         break;
130     }
131 out:
132     intr_restore(enabled);

```

```

133     if (allocated_slot < WORKFLOW_LIFECYCLE_CAP)
134         agent_identity_lease_lifecycle_note_next(
135             allocated_slot, next_generation);
136     return result;

```

角色规则不是用户态的字符串约定。内核按角色取得能力位掩码和调度权重，再在授权入口检查父角色、目标角色和能力位的合法关系。因此，工具、VFS、IPC 和调度器都依据同一份身份信息作出判断。

代码 3：角色策略表查询与调度权重取得

c

```

128 const struct agent_role_policy *
129 agent_identity_role_policy(int role)
130 {
131     for (uint i = 0; i < NELEM(role_policies); i++)
132         if (role_policies[i].role == role)
133             return &role_policies[i];
134     return 0;
135 }
136
137 int
138 agent_identity_role_valid(int role)
139 {
140     return agent_identity_role_policy(role) != 0;
141 }
142
143 int
144 agent_identity_role_sched_weight(int role)
145 {
146     const struct agent_role_policy *policy =
147         agent_identity_role_policy(role);
148
149     return policy ? policy->sched_weight : 50;
150 }

```

代码 4：父角色授权与进程访问范围校验

c

```

190 int
191 agent_identity_authority_check(struct proc *p, int role)
192 {
193     if (!agent_identity_role_valid(role))
194         return AGENT_STATUS_BAD_PARAM;
195     if (p == 0 ||
196         (p->is_agent ?
197          (!exec_policy_process_allows_role(p, p->agent_role) ||
198           !exec_policy_process_allows_role(p, role)) :
199          (!exec_policy_process_bootstrap(p) ||
200           !exec_policy_process_allows_role(p, role))) ||
201         (p->agent_role_grant_mask & AGENT_ROLE_GRANT_BIT(role)) == 0)
202         return AGENT_STATUS_DENIED;
203     return AGENT_STATUS_OK;
204 }
205
206 uint

```

```

207 agent_identity_proc_scope(struct proc *p)
208 {
209     if (p == 0 || !p->is_agent ||
210         p->agent_control_state != AGENT_CONTROL_OPEN ||
211         p->vfs_scope_id < VFS_SCOPE_FIRST_DYNAMIC ||
212         p->vfs_scope_id >= FS_OWNER_SCOPE_FLAG ||
213         !vfs_proc_lifecycle_active(p) ||
214         !vfs_scope_active(p->vfs_scope_id))
215         return VFS_SCOPE_NONE;
216     return p->vfs_scope_id;

```

算法 2：带代次的工作流进入检查

1. 内核从当前进程控制块读取工作流编号和代次，并验证 id 与 generation 均非零。
2. 内核在生命周期表中同时查找这两个字段；仅槽位相同不能通过校验。
3. 内核进入操作保护区后才执行系统副作用；屏障或回收已关闭该路径时返回重试或过期错误。
4. 异步对象在完成、取消或回收时再次比较代次，再释放生命周期引用。

3.8.2. 结构化工具调用与执行约定

V3 调用先检查用户内存范围、ABI 版本和大小、保留字段、参数个数和生命周期绑定。请求中的执行约定、节点、尝试号、参数规则摘要和输入指纹会复制到内核记录。因此，一次工具调用不再是孤立的函数调用，而是执行约定 DAG 中可核验的节点执行。

代码 5：V3 请求的 ABI、生命周期和执行约定绑定检查

c

```

2590     if (copyin(p->pagetable, (char *)&req, reqaddr, sizeof(req)) < 0 ||
2591         user_range_check(p->pagetable, respaddr, sizeof(resp), PTE_W) < 0)
2592         return -1;
2593     memset(&resp, 0, sizeof(resp));
2594     resp.version = AGENT_CALL_VERSION_V3;
2595     resp.size = sizeof(resp);
2596     resp.request_id = req.request_id;
2597     lifecycle = vfs_proc_lifecycle(p);
2598     resp.contract.lifecycle.id = lifecycle.id;
2599     resp.contract.lifecycle.reserved = 0;
2600     resp.contract.lifecycle.generation = lifecycle.generation;
2601     resp.contract.generation =
2602         agent_execution_contract_generation(lifecycle);
2603     resp.node_id = req.node_id;
2604     resp.attempt_id = req.attempt_id;
2605     memmove(resp.input_fingerprint, req.input_fingerprint,
2606             sizeof(resp.input_fingerprint));
2607     resp.source_context_sequence = req.source_context_sequence;
2608     memset(&binding, 0, sizeof(binding));
2609     binding.lifecycle.id = req.contract.lifecycle.id;
2610     binding.lifecycle.generation = req.contract.lifecycle.generation;
2611     binding.contract_generation = req.contract.generation;
2612     binding.node_id = req.node_id;
2613     binding.attempt_id = req.attempt_id;
2614     memmove(binding.input_fingerprint, req.input_fingerprint,

```

```

2615     sizeof(binding.input_fingerprint));
2616     binding.source_context_sequence = req.source_context_sequence;
2617     binding.source_control_id = req.source_control_id;
2618     binding.source_pid = req.source_pid;
2619     binding.source_reserved = req.source_reserved;
2620     memmove(binding.schema_digest, req.schema_digest,
2621             sizeof(binding.schema_digest));
2622     binding.input_artifact_type = req.input_artifact_type;
2623     binding.source_node_id = req.source_node_id;
2624     binding.input_mode = AGENT_EXECUTION_INPUT_INLINE;
2625     memset(&op, 0, sizeof(op));
2626     op.version = AGENT_CALL_VERSION_V3;
2627     op.tool_id = req.tool_id > 0 ? req.tool_id :
2628         (0x20000000 | (int)((req.node_id + 1) & 0xffffU));
2629     op.request_id = req.request_id;
2630     op.arg0 = req.contract.generation;
2631     op.arg1 = ((uint64)req.node_id << 32) | req.attempt_id;
2632     safestrncpy(op.payload, "v3_rejected", sizeof(op.payload));
2633     enforced = p->is_agent &&
2634         agent_execution_contract_enforced(lifecycle);
2635     memset(error, 0, sizeof(error));
2636     if (req.version != AGENT_CALL_VERSION_V3) {
2637         status = AGENT_STATUS_BAD_VERSION;
2638         safestrncpy(error, "bad_request_version", sizeof(error));
2639     } else if (req.size != sizeof(req)) {
2640         status = AGENT_STATUS_BAD_SIZE;
2641         safestrncpy(error, "bad_request_size", sizeof(error));
2642     } else if (req.flags != 0 || req.source_reserved != 0) {
2643         status = AGENT_STATUS_BAD_PARAM;
2644         safestrncpy(error, "unsupported_fields", sizeof(error));
2645     } else if (req.param_count > AGENT_TOOL_PARAM_MAX) {
2646         status = AGENT_STATUS_BAD_SIZE;
2647         safestrncpy(error, "too_many_params", sizeof(error));
2648     } else if (req.contract.lifecycle.id == 0 ||
2649         req.contract.lifecycle.generation == 0 ||
2650         req.contract.generation == 0) {
2651         status = AGENT_STATUS_BAD_PARAM;
2652         safestrncpy(error, "bad_contract_binding", sizeof(error));
2653     } else if (!p->is_agent) {
2654         status = AGENT_STATUS_NOT_AGENT;
2655         safestrncpy(error, "not_agent", sizeof(error));
2656     } else if (req.contract.lifecycle.reserved != 0 ||
2657         req.contract.lifecycle.id != lifecycle.id ||
2658         req.contract.lifecycle.generation != lifecycle.generation) {
2659         status = AGENT_STATUS_STALE;
2660         denial_reason = AGENT_EXECUTION_REASON_STALE_LIFECYCLE;
2661         safestrncpy(error, "stale_lifecycle", sizeof(error));
2662     } else {
2663         status = agent_tool_protocol_resolve(req.tool_id, req.tool_name,
2664             &tool, error, sizeof(error));
2665         if (status != AGENT_STATUS_OK)
2666             denial_reason = AGENT_EXECUTION_REASON_TOOL_MISMATCH;
2667     }

```

解析阶段先按工具名称或编号查找定义，再逐项从用户内存读取带类型信息的参数。每个参数都必须符合目录规则规定的名称、类型、长度和唯一性，最后还要确认必填参数

全部出现。用户态即使构造出形式正确的结构，也不能绕过内核目录中的参数规则。

代码 6: V2 参数规则解码器

c

```

548 int agent_tool_protocol_decode_v2(pagetable_t pagetable,
549     struct agent_request_v2 *request,
550     struct agent_tool_match *match,
551     struct agent_op *op, char *error,
552     int error_size)
553 {
554     uint seen = 0, count = rule_count(match->tool_id);
555     const struct param_rule *tool_rules = rules_for_tool(match->tool_id);
556
557     memset(op, 0, sizeof(*op));
558     op->version = AGENT_CALL_VERSION_V1;
559     op->tool_id = match->tool_id;
560     op->request_id = request->request_id;
561     for (uint i = 0; i < request->param_count; i++) {
562         struct agent_param_v2 param;
563         uint text_length = 0;
564         int index = -1;
565
566         if (copyin(pagetable, (char *)&param,
567             request->params + (uint64)i * sizeof(param),
568             sizeof(param)) < 0)
569             return -1;
570         if (param.version != AGENT_PARAM_VERSION)
571             REJECT(AGENT_STATUS_BAD_VERSION, "bad_param_version");
572         if (param.size != sizeof(param) ||
573             string_length(param.key, AGENT_PARAM_KEY_SIZE, 0) < 0)
574             REJECT(AGENT_STATUS_BAD_SIZE, "bad_param_size");
575         for (uint j = 0; j < count; j++)
576             if (!strcmp(param.key, rule_key(&tool_rules[j]),
577                 AGENT_PARAM_KEY_SIZE)) {
578                 index = j;
579                 break;
580             }
581         if (index < 0)
582             REJECT(AGENT_STATUS_UNKNOWN_PARAM, "unknown_param");
583         if (seen & (1U << index))
584             REJECT(AGENT_STATUS_DUPLICATE, "duplicate_param");
585         seen |= 1U << index;
586         if (param.type != rule_type(&tool_rules[index]))
587             REJECT(AGENT_STATUS_BAD_TYPE, "bad_param_type");
588         if (param.type == AGENT_PARAM_UINT64) {
589             if (param.value_size != sizeof(param.value.uint64_value))
590                 REJECT(AGENT_STATUS_BAD_SIZE, "bad_value_size");
591         } else if (param.value_size == 0 ||
592             param.value_size > AGENT_PARAM_STRING_SIZE ||
593             string_length(param.value.string_value, param.value_size,
594                 &text_length) < 0 ||
595             text_length + 1 != param.value_size)
596             REJECT(AGENT_STATUS_BAD_SIZE, "bad_value_size");
597         switch (rule_target(&tool_rules[index])) {
598             case PARAM_ARG0: op->arg0 = param.value.uint64_value; break;
599             case PARAM_ARG1: op->arg1 = param.value.uint64_value; break;

```



```

600     default: safestrcpy(op->payload, param.value.string_value,
601                         sizeof(op->payload)); break;
602     }
603 }
604 for (uint i = 0; i < count; i++)
605     if (rule_required(&tool_rules[i]) && !(seen & (1U << i)))
606         REJECT(AGENT_STATUS_BAD_PARAM, "missing_param");
607 return AGENT_STATUS_OK;

```

参数校验通过后，调用先进入生命周期操作保护区，再调用 `agent_execute_one()`。该入口汇总调用结果、输出任务产物类型、拒绝原因、来源标签和执行记录票号；受规则约束的拒绝会尝试写入安全记录。关键记录无法预留或提交时，直接调用和 V3 调用的拒绝会返回 `NO_SPACE` 或 `RETRY`，因此不是每次拒绝都会得到执行记录票号。

代码 7：V3 调用的解码、检查与执行进入

c

```

2678     if ((req.param_count == 0 && req.params != 0) ||
2679         (req.param_count != 0 && req.params == 0)) {
2680         status = AGENT_STATUS_BAD_PARAM;
2681         safestrcpy(error, "bad_param_pointer", sizeof(error));
2682         goto rejected;
2683     }
2684     if (req.param_count != 0 &&
2685         user_range_check(p->pagetable, req.params,
2686                         (uint64)req.param_count *
2687                         sizeof(struct agent_param_v2), PTE_R) < 0) {
2688         status = AGENT_STATUS_DENIED;
2689         safestrcpy(error, "bad_param_range", sizeof(error));
2690         goto rejected;
2691     }
2692     memset(&decode_request, 0, sizeof(decode_request));
2693     decode_request.version = AGENT_CALL_VERSION_V2;
2694     decode_request.size = sizeof(decode_request);
2695     decode_request.tool_id = req.tool_id;
2696     decode_request.param_count = req.param_count;
2697     decode_request.request_id = req.request_id;
2698     decode_request.params = req.params;
2699     safestrcpy(decode_request.tool_name, req.tool_name,
2700               sizeof(decode_request.tool_name));
2701     status = agent_tool_protocol_decode_v2(
2702         p->pagetable, &decode_request, &tool, &op, error, sizeof(error));
2703     if (status != AGENT_STATUS_OK) {
2704         status = AGENT_STATUS_DENIED;
2705         denial_reason = AGENT_EXECUTION_REASON_CONTRACT_INVALID;
2706         goto rejected;
2707     }
2708     if (workflow_lifecycle_operation_enter(lifecycle) < 0) {
2709         status = AGENT_STATUS_RETRY;
2710         safestrcpy(error, "workflow_fenced", sizeof(error));
2711         goto out;
2712     }
2713     operation_entered = 1;
2714     if (agent_execute_one(p, &op, &result, agent_ticks(), &binding, 0, 0,
2715                          &outcome) < 0)
2716         result.status = AGENT_STATUS_NO_SPACE;

```

```

2717     status = result.status;
2718     resp.sequence = result.sequence;
2719     resp.value0 = result.value0;
2720     resp.value1 = result.value1;
2721     resp.value2 = result.value2;
2722     resp.output_artifact_type = outcome.output_artifact_type;
2723     resp.decision_reason = outcome.decision_reason;
2724     resp.completion_flags = outcome.completion_flags;
2725     resp.output_provenance_labels = outcome.output_provenance_labels;
2726     resp.evidence_ticket = outcome.evidence_ticket;
2727     safestrcpy(resp.result, result.result, sizeof(resp.result));
2728     goto out;

```

代码 8：受控拒绝的安全记录与返回封装

c

```

2730 rejected:
2731     if (enforced && op.tool_id > 0) {
2732         if (op.request_id == 0) {
2733             /* Keep user correlation zero while giving Evidence a valid key. */
2734             op.request_id = (1ULL << 63) |
2735                 ((uint64) (uint)p->pid << 32) |
2736                 (uint) (p->agent_call_count + 1);
2737         }
2738         status = agent_tool_call_v3_security_denial(
2739             p, &binding, &op, status, denial_reason,
2740             &result, &outcome);
2741         resp.sequence = result.sequence;
2742         resp.output_artifact_type = outcome.output_artifact_type;
2743         resp.decision_reason = outcome.decision_reason;
2744         resp.output_provenance_labels =
2745             outcome.output_provenance_labels;
2746         resp.evidence_ticket = outcome.evidence_ticket;
2747         safestrcpy(resp.result, result.result, sizeof(resp.result));
2748         error[0] = 0;
2749     }
2750 out:
2751     if (operation_entered)
2752         workflow_lifecycle_operation_leave(lifecycle);
2753     if (enforced &&
2754         (status == AGENT_STATUS_DENIED || status == AGENT_STATUS_STALE) &&
2755         resp.evidence_ticket == 0) {
2756         status = AGENT_STATUS_RETRY;
2757         error[0] = 0;
2758         safestrcpy(resp.result, "security_evidence_unavailable",
2759             sizeof(resp.result));
2760     }
2761     resp.status = status;
2762     if (status != AGENT_STATUS_OK && error[0] != 0)
2763         safestrcpy(resp.result, error, sizeof(resp.result));
2764     return copyout(p->pagetable, respaddr, (char *)&resp, sizeof(resp));

```

执行约定的接纳检查会逐项比对绑定的生命周期、执行约定代次、节点编号、截止时间、工具编号和资源额度。任务通道的内部请求还限制为“不产生输出”的预检形式；被拒绝的任务不会因此发布伪造的带类型信息输出。

代码 9：执行约定节点的代次、资源与工具一致性检查

c

```

1157     if (!workflow_lifecycle_key_equal(binding->lifecycle, current)) {
1158         agent_execution_result_error(
1159             result, AGENT_STATUS_STALE, "stale_lifecycle",
1160             AGENT_EXECUTION_REASON_STALE_LIFECYCLE, claim);
1161         goto denied_counted;
1162     }
1163     if (!agent_execution_record_enforced(record, current)) {
1164         agent_execution_result_error(
1165             result, AGENT_STATUS_DENIED, "execution_contract_missing",
1166             AGENT_EXECUTION_REASON_CONTRACT_REQUIRED, claim);
1167         goto denied_counted;
1168     }
1169     if (record->state == AGENT_EXECUTION_CONTRACT_RETIRING) {
1170         agent_execution_result_error(
1171             result,
1172             (binding->internal_flags &
1173              AGENT_EXECUTION_BINDING_INTERNAL_F_TASK_CHANNEL) != 0 ?
1174             AGENT_STATUS_DENIED : AGENT_STATUS_CANCELLED,
1175             "contract_retiring",
1176             AGENT_EXECUTION_REASON_CONTRACT_RETIRING, claim);
1177         goto denied_counted;
1178     }
1179     if (binding->contract_generation == 0 ||
1180         binding->contract_generation != record->generation) {
1181         agent_execution_result_error(
1182             result, AGENT_STATUS_STALE, "stale_contract",
1183             AGENT_EXECUTION_REASON_STALE_CONTRACT, claim);
1184         goto denied_counted;
1185     }
1186     if (binding->node_id >= record->node_count) {
1187         agent_execution_result_error(
1188             result, AGENT_STATUS_DENIED, "unknown_contract_node",
1189             AGENT_EXECUTION_REASON_UNKNOWN_NODE, claim);
1190         goto denied_counted;
1191     }
1192     node = &record->nodes[binding->node_id];
1193     runtime = &record->runtime[binding->node_id];
1194     claim->output_artifact_type = node->output_artifact_type;
1195     claim->charge_class = node->charge_class;
1196     contract_deadline = node->deadline_tick != 0 &&
1197         (record->deadline_tick == 0 ||
1198          node->deadline_tick < record->deadline_tick) ?
1199         node->deadline_tick : record->deadline_tick;
1200     claim->deadline_tick = request_deadline_tick != 0 ?
1201         request_deadline_tick : contract_deadline;
1202     claim->manifest.accepted_input_labels = node->accepted_input_labels;
1203     claim->manifest.output_add_labels = node->output_add_labels;
1204     claim->manifest.required_capabilities = node->required_capabilities;
1205     claim->manifest.side_effect_mask = node->side_effect_mask;
1206     for (uint kind = 0; kind < RESOURCE_KIND_COUNT; kind++) {
1207         claim->exec_amounts[kind] = node->exec_envelope[kind];
1208         claim->storage_amounts[kind] = node->storage_envelope[kind];
1209     }
1210     if ((binding->internal_flags &

```

```

1211     AGENT_EXECUTION_BINDING_INTERNAL_F_TASK_CHANNEL) != 0) {
1212     uint expected_policy =
1213         AGENT_EXECUTION_PREFLIGHT_F_OUTPUT_NONE_ONLY |
1214         (request_deadline_tick != 0 ?
1215          AGENT_EXECUTION_PREFLIGHT_F_HARD_DEADLINE : 0);
1216
1217     if (admission_policy_flags != expected_policy ||
1218         (contract_deadline != 0 &&
1219          (admission_policy_flags &
1220           AGENT_EXECUTION_PREFLIGHT_F_HARD_DEADLINE) == 0) ||
1221         node->output_artifact_type != AGENT_ARTIFACT_NONE) {
1222         /* A rejected Task request never publishes a typed output. */
1223         claim->output_artifact_type = AGENT_ARTIFACT_NONE;
1224         agent_execution_result_error(
1225             result, AGENT_STATUS_DENIED,
1226             "task_admission_policy",
1227             AGENT_EXECUTION_REASON_CONTRACT_INVALID, claim);
1228         goto denied_counted;
1229     }
1230 } else if (admission_policy_flags != 0) {
1231     agent_execution_result_error(
1232         result, AGENT_STATUS_DENIED,
1233         "unexpected_admission_policy",
1234         AGENT_EXECUTION_REASON_CONTRACT_INVALID, claim);
1235     goto denied_counted;
1236 }
1237 if ((runtime->flags &
1238     AGENT_EXECUTION_RUNTIME_F_DEPENDENCY_TERMINAL) != 0) {
1239     claim->dependency_failed = 1;
1240     claim->retry_forbidden = 1;
1241     claim->decision_reason =
1242         AGENT_EXECUTION_REASON_DEPENDENCY_FAILED;
1243 }
1244 if (node->tool_id != (uint)op->tool_id) {
1245     agent_execution_result_error(
1246         result, AGENT_STATUS_DENIED, "contract_tool_mismatch",
1247         AGENT_EXECUTION_REASON_TOOL_MISMATCH, claim);
1248     goto denied_counted;
1249 }
1250 if (!agent_execution_digest_equal(binding->schema_digest,

```

真正触发副作用之前，内核再次确认节点仍在运行，请求号、尝试号、摘要与当前声明一致，并处理超时或取消的胜出分支。这个检查缩短了“执行约定通过”和“副作用实际发生”之间的时间差，避免状态过期后继续产生内核可见副作用。

代码 10：副作用开始前的执行约定检查

c

```

1781 enum agent_execution_effect_admission
1782 agent_execution_contract_effect_begin(struct agent_execution_claim *claim)
1783 {
1784     struct agent_execution_contract_record *record;
1785     struct agent_execution_node_runtime *runtime;
1786     int enabled;
1787
1788     if (claim == 0 || !claim->active)

```

```

1789     return AGENT_EXECUTION_EFFECT_STALE;
1790     if (claim->legacy)
1791         return AGENT_EXECUTION_EFFECT_ALLOWED;
1792     if (claim->slot < 0 || claim->slot >= WORKFLOW_LIFECYCLE_CAP ||
1793         claim->node_id >= AGENT_EXECUTION_CONTRACT_MAX_NODES)
1794         return AGENT_EXECUTION_EFFECT_STALE;
1795     enabled = intr_save();
1796     record = &agent_execution_contracts[claim->slot];
1797     if (!agent_execution_record_matches(record, claim->lifecycle) ||
1798         record->generation != claim->contract_generation ||
1799         claim->node_id >= record->node_count) {
1800         intr_restore(enabled);
1801         return AGENT_EXECUTION_EFFECT_STALE;
1802     }
1803     runtime = &record->runtime[claim->node_id];
1804     if (runtime->state != AGENT_EXECUTION_NODE_RUNNING ||
1805         runtime->request_id != claim->request_id ||
1806         runtime->attempt_id != claim->attempt_id ||
1807         !agent_execution_digest_equal(runtime->request_digest,
1808                                       claim->request_digest)) {
1809         intr_restore(enabled);
1810         return AGENT_EXECUTION_EFFECT_STALE;
1811     }
1812     if ((runtime->flags &
1813         AGENT_EXECUTION_RUNTIME_F_TIMEOUT_REQUESTED) != 0) {
1814         claim->decision_reason =
1815             AGENT_EXECUTION_REASON_DEADLINE_EXPIRED;
1816         intr_restore(enabled);
1817         return AGENT_EXECUTION_EFFECT_STALE;
1818     }
1819     if ((runtime->flags &
1820         AGENT_EXECUTION_RUNTIME_F_CANCEL_REQUESTED) != 0) {
1821         claim->decision_reason =
1822             AGENT_EXECUTION_REASON_CANCEL_REQUESTED;
1823         if ((runtime->flags &
1824             AGENT_EXECUTION_RUNTIME_F_FORCE_CANCEL) != 0)
1825             claim->retry_forbidden = 1;
1826         intr_restore(enabled);
1827         return AGENT_EXECUTION_EFFECT_CANCELLED;
1828     }
1829     runtime->flags |= AGENT_EXECUTION_RUNTIME_F_EFFECT_STARTED;
1830     intr_restore(enabled);
1831     return AGENT_EXECUTION_EFFECT_ALLOWED;

```

算法 3：一次 V3 工具调用的内核决策顺序

1. 内核验证固定大小 ABI、用户地址、智能体身份和工作流代次。
2. 内核查找工具描述符并按参数规则解码；失败时构造结构化拒绝结果，关键记录无法建立时返回 NO_SPACE 或 RETRY。
3. 接纳检查核对执行约定 DAG、输入来源、能力位、截止时间和资源额度。
4. 开始副作用前，内核再次确认当前声明；执行后写入上下文、执行记录、来源信息和完成结果。

3.8.3. 上下文、回滚边界与数据来源

上下文附属记录只保存归一化的 agent_op、结果和归因，不完整保留 V2 参数、V3 绑定或任务 SQE。镜像页按 prepare--begin--end 顺序发布；detail 在记录标为奇数前写入，序号只在一次清空后的时期内单调增加，清空后从 1 重新开始。因此，客户机看到的只读页是稳定快照，而不是写到一半的记录。

代码 11：上下文附属记录的写入与发布窗口

c

```

1152 static int
1153 agent_context_append_flags(struct proc *p, struct agent_op *op,
1154                          struct agent_result *latest, uint64 tick,
1155                          uint64 flags, int authority_effect,
1156                          int causal_audit,
1157                          struct agent_evidence_context_reservation *reservation,
1158                          struct agent_evidence_security_reservation *
1159                          security_reservation,
1160                          uint64 *evidence_ticket_out)
1161 {
1162     struct agent_context_append_receipt receipt;
1163     struct agent_context_detail detail;
1164     struct agent_context_record record;
1165     struct thread *t = curr_thread();
1166     uint64 slot;
1167
1168     if (p == 0 || op == 0 || latest == 0 || p->agent_ctx_base == 0 ||
1169         p->context_path_capacity == 0 || !agent_context_layout_ok() ||
1170         p->agent_current_span_id == 0 ||
1171         p->agent_current_span_owner == 0 || t == 0 || t->process != p ||
1172         p->agent_context_lane_owner_tid != t->tid ||
1173         p->agent_context_lane_depth == 0 ||
1174         latest->sequence != p->agent_call_count)
1175         return -1;
1176     if ((reservation != 0 && security_reservation != 0) ||
1177         ((reservation != 0 || security_reservation != 0) &&
1178          (evidence_ticket_out == 0 ||
1179           agent_observe_recording_suppressed(p))) ||
1180         (reservation != 0 && !reservation->active) ||
1181         (security_reservation != 0 && !security_reservation->active))
1182         return -1;
1183     slot = p->context_path_head % p->context_path_capacity;
1184
1185     memset(&record, 0, sizeof(record));
1186     record.sequence = latest->sequence;
1187     record.request_id = latest->request_id;
1188     record.cause_sequence = p->agent_current_cause_sequence;
1189     record.span_id = p->agent_current_span_id;
1190     record.branch_generation = p->context_branch_generation;
1191     record.path_parent_sequence = p->context_path_visible_head;
1192     record.arg0 = op->arg0;
1193     record.value0 = latest->value0;
1194     record.value1 = latest->value1;
1195     record.value2 = latest->value2;
1196     record.tick = tick;
1197     record.flags = agent_provenance_context_flags(

```



```

1198     p, latest->request_id, latest->tool_id, flags);
1199     if (strlen(op->payload) >= sizeof(record.payload) ||
1200         strlen(latest->result) >= sizeof(record.result))
1201         record.flags |= AGENT_CONTEXT_RECORD_F_TRUNCATED;
1202     record.tool_id = latest->tool_id;
1203     record.status = latest->status;
1204     safestrncpy(record.payload, op->payload, sizeof(record.payload));
1205     safestrncpy(record.result, latest->result, sizeof(record.result));
1206     record.prev_hash = p->agent_context_chain_hash;
1207     record.record_hash = agent_context_record_hash(&record);
1208     if (agent_context_append_receipt_prepare(p, &record, slot, &receipt) < 0)
1209         return -1;
1210
1211     memset(&detail, 0, sizeof(detail));
1212     detail.sequence = latest->sequence;
1213     detail.flags = record.flags;
1214     memmove(&detail.op, op, sizeof(*op));
1215     memmove(&detail.result, latest, sizeof(*latest));
1216     if (agent_context_store(p, slot, &detail,
1217         p->agent_current_span_owner,
1218         p->agent_current_cause_pid,
1219         p->agent_current_cause_control) < 0)
1220         return -1;
1221     agent_context_publish_begin(p);
1222     agent_context_write_record(p, slot, &record);
1223     if (p->context_path_count < p->context_path_capacity) {
1224         if (p->context_path_count == 0)
1225             p->context_path_oldest = latest->sequence;
1226         p->context_path_count++;
1227     } else {
1228         p->context_path_oldest =
1229             latest->sequence - p->context_path_capacity + 1;
1230     }
1231     p->context_path_latest = latest->sequence;
1232     p->context_path_head = (slot + 1) % p->context_path_capacity;
1233     if (record.cause_sequence != 0)
1234         p->agent_provenance_edges++;
1235     if (agent_context_append_receipt_commit(p, &record, &receipt) < 0)
1236         panic("Agent context append receipt");
1237     p->agent_context_chain_hash = record.record_hash;
1238     p->agent_current_cause_sequence = latest->sequence;
1239     p->context_cause_branch_generation = record.branch_generation;
1240     p->agent_current_cause_pid = p->pid;
1241     p->agent_current_cause_control = p->agent_control_id;
1242     p->agent_current_span_id = record.span_id;
1243     agent_provenance_context_committed(
1244         p, record.request_id, record.tool_id, record.flags);
1245     if (agent_context_write_latest(p, latest) < 0 ||
1246         agent_context_write_header_locked(p) < 0)
1247         panic("Agent context direct publish");
1248     /* Reserved terminal and denial records stay behind an odd sequence until
1249      * their canonical Evidence ticket is committed. The release below is the
1250      * only point at which direct readers may accept the new Context snapshot. */
1251     if (reservation != 0)
1252         *evidence_ticket_out =
1253             agent_observe_commit_context_reserved_ticket(
1254                 p, &record, authority_effect, causal_audit,

```

```

1255         reservation);
1256     else if (security_reservation != 0)
1257         *evidence_ticket_out =
1258             agent_observe_commit_security_reserved_ticket(
1259                 p, &record, security_reservation);
1260     agent_context_publish_end(p);
1261     if (reservation != 0 || security_reservation != 0) {
1262         agent_observe_publish_context_ticket(
1263             p, &record, authority_effect, causal_audit,
1264             *evidence_ticket_out);

```

执行入口在执行约定接纳之前为上下文追加记录做准备；输入来源标签随执行约定声明合并到当前进程。面对截止时间、依赖失败和策略拒绝，代码会尝试生成完成记录或安全记录，让调试和测试区分“未执行”“拒绝”和“执行后失败”。执行记录无法建立时，直接调用和 V3 调用的拒绝仍以 NO_SPACE 或 RETRY 表达失败。手工注记、回滚和清空不使用执行记录的预留与提交票号规则。

代码 12：执行入口中的上下文准备与来源合并

c

```

1577 agent_execute_one(struct proc *p, struct agent_op *op,
1578                 struct agent_result *res, uint64 tick,
1579                 const struct agent_execution_binding *binding,
1580                 uint64 request_deadline_tick, uint admission_policy_flags,
1581                 struct agent_execution_outcome *outcome)
1582 {
1583     struct agent_execution_claim claim;
1584     struct agent_provenance_request provenance_request;
1585     struct agent_provenance_decision provenance_decision;
1586     struct agent_evidence_context_reservation evidence_reservation = { 0 };
1587     struct resource_phase_lease phase_lease = { 0 };
1588     struct resource_account_handle storage = resource_account_none();
1589     enum agent_execution_admission admission;
1590     enum agent_execution_effect_admission effect_admission;
1591     int bound = binding != 0;
1592     int phase_started = 0;
1593     uint64 phase_generation = 0;
1594     uint64 terminal_tick = tick;
1595     int result = 0;
1596     int status;
1597
1598     if (outcome != 0)
1599         memset(outcome, 0, sizeof(*outcome));
1600     op->payload[AGENT_OP_PAYLOAD_SIZE - 1] = 0;
1601     agent_result_init(res, op);
1602     if (agent_lifecycle_context_lane_enter(p) < 0) {
1603         res->status = AGENT_STATUS_NO_SPACE;
1604         agent_result_text(res, "context_interrupted");
1605         return -1;
1606     }
1607     if (agent_context_append_prepare(p, p->agent_call_count + 1) < 0) {
1608         res->status = AGENT_STATUS_NO_SPACE;
1609         agent_result_text(res, "context_sync_unavailable");
1610         agent_lifecycle_context_lane_leave(p);
1611         return -1;

```

```

1612     }
1613     admission = agent_execution_contract_admit(
1614         p, binding, op, request_deadline_tick, admission_policy_flags,
1615         tick, &claim, res, outcome);
1616     if (bound && admission == AGENT_EXECUTION_ADMISSION_EXECUTE &&
1617         claim.input_provenance_labels != 0 &&
1618         agent_provenance_merge_current(
1619             p, claim.input_provenance_labels) != AGENT_STATUS_OK)
1620         panic("contract input provenance merge");
1621     if (outcome != 0 && admission != AGENT_EXECUTION_ADMISSION_CACHED) {
1622         outcome->output_artifact_type = claim.output_artifact_type;
1623         outcome->terminal_tick = terminal_tick;
1624     }
1625     if (admission == AGENT_EXECUTION_ADMISSION_DENIED) {
1626         if (res->status == AGENT_STATUS_DENIED ||
1627             res->status == AGENT_STATUS_STALE) {
1628             result = agent_execution_security_denial(
1629                 p, &claim, op, bound, res, outcome);
1630             agent_execution_publish_policy_denial(
1631                 p, op, res, &claim);
1632             goto out;
1633         }
1634         p->agent_call_count++;
1635         res->sequence = p->agent_call_count;
1636         result = agent_context_append(p, op, res, tick, 0);
1637         if (result < 0)
1638             p->agent_call_count--;
1639         goto out;
1640     }
1641     if (admission == AGENT_EXECUTION_ADMISSION_CACHED) {
1642         if (outcome != 0)
1643             outcome->completion_flags |= AGENT_RESPONSE_V3_F_CACHED;
1644         goto out;
1645     }
1646     if (claim.deadline_expired || claim.dependency_failed) {
1647         if (agent_evidence_context_reserve(
1648             p, &evidence_reservation) < 0) {
1649             res->status = AGENT_STATUS_RETRY;
1650             agent_result_text(res, "evidence_reservation_unavailable");
1651             agent_execution_contract_abort(&claim);
1652             goto out;
1653         }
1654         p->agent_call_count++;
1655         res->sequence = p->agent_call_count;
1656         res->status = claim.deadline_expired ?
1657             AGENT_STATUS_TIMEOUT : AGENT_STATUS_CANCELLED;
1658         agent_result_text(
1659             res, claim.deadline_expired ?
1660                 "contract_deadline" : "dependency_failed");
1661         if (outcome != 0) {
1662             outcome->decision_reason = claim.decision_reason;
1663             outcome->output_provenance_labels =
1664                 agent_provenance_current_labels(p);
1665         }
1666         result = agent_execution_append_terminal(
1667             p, op, res, tick, 0, &claim, outcome,
1668             &evidence_reservation);

```

回滚接口只恢复上下文路径的可见记录和分支状态，不会撤销已经发生的文件写入、消息发送或外部服务调用。因此，上下文回滚用于修正后续推理和可见因果关系，不承担通用事务回滚的职责。

3.8.4. 元数据查询与实时查询事件

文件查询先确定执行方式：调用者可以明确要求遍历、关闭索引，或者在启用索引时让内核按固定的 `status->stage->kind` 顺序选择第一个给定字段。选择时不比较基数。随后，内核在目录读取快照中扫描候选记录，检查访问范围和筛选条件，并定期让出内核工作预算。结果同时返回命中总数、实际返回数、truncated、是否使用索引、原因、查询时钟数和文件系统代次，供用户态解释本次查询。

代码 13：元数据查询的索引计划与快照执行

c

```

143 static void
144 agent_query_plan_build(const struct agent_file_query *q,
145                       struct agent_file_query_result *r,
146                       struct agent_query_plan *plan)
147 {
148     memset(plan, 0, sizeof(*plan));
149     plan->limit = q->max_hits;
150     if (plan->limit <= 0 || plan->limit > AGENT_FILE_QUERY_MAX_HITS)
151         plan->limit = AGENT_FILE_QUERY_MAX_HITS;
152     r->plan = AGENT_FILE_QUERY_PLAN_SCAN;
153     r->index_bucket = -1;
154     if (q->flags & AGENT_FILE_QUERY_SCAN) {
155         plan->force_scan = 1;
156         plan->reason |= AGENT_FILE_QUERY_REASON_FORCED_SCAN;
157         return;
158     }
159     if ((q->flags & AGENT_FILE_QUERY_USE_INDEX) == 0) {
160         plan->reason |= AGENT_FILE_QUERY_REASON_INDEX_OFF;
161         return;
162     }
163     for (int i = AGENT_QUERY_INDEX_FIRST; i < AGENT_QUERY_FIELDS; i++) {
164         const struct agent_query_alias *alias = &agent_query_aliases[i];
165         const char *value = (const char *)q + alias->offset;
166
167         if (!value[0])
168             continue;
169         plan->index = AGENT_CATALOG_INDEX_STATUS +
170             i - AGENT_QUERY_INDEX_FIRST;
171         plan->use_index = 1;
172         plan->index_key = value;
173         r->plan = AGENT_FILE_QUERY_PLAN_STATUS_INDEX +
174             i - AGENT_QUERY_INDEX_FIRST;
175         plan->reason |= AGENT_FILE_QUERY_REASON_STATUS_INDEX <<
176             (i - AGENT_QUERY_INDEX_FIRST);
177         return;
178     }
179     plan->reason |= AGENT_FILE_QUERY_REASON_NO_INDEX_KEY;
180 }
181
182 static void agent_query_result_reset(struct agent_file_query_result *r)

```

```

183 {
184     memset(r, 0, sizeof(*r));
185 }
186
187 int
188 agent_metadata_query_execute_snapshot(
189     uint scope, const struct agent_file_query *q,
190     struct agent_file_query_result *r)
191 {
192     struct agent_catalog_read_snapshot snapshot;
193     struct agent_file_meta meta;
194     struct agent_query_plan plan;
195     uint owner;
196     uint work = 0;
197     int bucket = -1;
198     int copied;
199     uint64 start;
200
201     agent_query_result_reset(r);
202     agent_query_plan_build(q, r, &plan);
203     start = agent_file_state_now();
204     copied = agent_metadata_catalog_read_begin(
205         scope, plan.index, plan.index_key, plan.force_scan,
206         &snapshot, &bucket);
207     if (copied < 0)
208         return copied;
209     if (plan.use_index)
210         r->index_bucket = bucket;
211     for (int slot = agent_metadata_catalog_read_next(&snapshot, -1);
212          slot >= 0;
213          slot = agent_metadata_catalog_read_next(&snapshot, slot)) {
214         r->scanned_records++;
215         copied = agent_metadata_catalog_read_copy(
216             &snapshot, slot, &meta, &owner);
217         if (copied < 0)
218             return copied;
219         if (copied > 0 && agent_object_scope_visible(scope, owner)) {
220             r->candidate_records++;
221             if (agent_metadata_query_matches(scope, owner, q, &meta)) {
222                 r->total_hits++;
223                 if (r->returned < plan.limit) {
224                     agent_file_state_project_hit(
225                         &r->hits[r->returned++],
226                         &meta, owner);
227                 } else {
228                     r->truncated = 1;
229                 }
230             }
231         }
232         if (++work == AGENT_QUERY_READ_GRANULE) {
233             work = 0;
234             (void)kernel_work_checkpoint_cleanup(
235                 KERNEL_WORK_OPERATION_UNITS);
236         }
237     }
238     if (!agent_metadata_catalog_read_end(&snapshot))
239         return AGENT_CATALOG_STALE;

```

```

240     r->used_index = plan.use_index;
241     r->index_rebuild_records = 0;
242     r->query_ticks = agent_file_state_now() - start;
243     r->plan_reason = plan.reason;
244     r->fs_generation = snapshot.fs_generation;
245     return r->returned;

```

实时查询根据元数据变化前后的状态生成 ENTER、UPDATE、LEAVE 三类事件。订阅只保存筛选条件和代次，不保存结果集；每个对象都会先检查工作流和访问范围是否可见。只有投递失败或队列出现缺口才需要重新同步，正常的文件系统代次变化仍持续投递三类增量。安全恢复时，智能体保留旧订阅，先按相同条件安装不带 ACK_RESYNC 的替代订阅，取得 truncated=0 的有界快照后，才在旧订阅上确认缺口并取消订阅；替代订阅覆盖切换窗口。 workflow 屏障会排空延迟工作；只要 workflow 或访问范围内的黏性重新同步尚未确认，即使标记已投递也返回 RETRY，最后的回收才会清理残留。

代码 14：实时查询的 ENTER、UPDATE、LEAVE 事件投递

c

```

1250     struct workflow_lifecycle_key source_key =
1251         agent_live_query_proc_key(source);
1252     uint source_scope = agent_identity_proc_scope(source);
1253     uint64 fid;
1254     int delivered = 0;
1255
1256     if (!agent_live_query_domain_valid(key, owner_scope) ||
1257         identity == 0 || !identity->used || identity->fid <= 0)
1258         return 0;
1259     if (generation == 0)
1260         generation = agent_file_state_scope_generation(owner_scope);
1261     fid = (uint64)(uint)identity->fid;
1262     for (struct proc *target = pool; target < &pool[NPROC]; target++) {
1263         struct workflow_lifecycle_key target_key;
1264         struct proc *attributed = 0;
1265         uint target_scope;
1266         uint legacy_changes;
1267         uint typed_changes;
1268         int enabled = intr_save();
1269
1270         if (!agent_live_query_target_visible(target, key, owner_scope) ||
1271             !agent_live_query_target_valid(
1272                 target, &target_key, &target_scope)) {
1273             intr_restore(enabled);
1274             continue;
1275         }
1276         legacy_changes = agent_live_query_target_changes(
1277             target, owner_scope, before, after);
1278         typed_changes = agent_live_query_typed_target_changes(
1279             target, owner_scope, before, after);
1280         if (legacy_changes == 0 && typed_changes == 0) {
1281             intr_restore(enabled);
1282             continue;
1283         }
1284         if (source != 0 && source_scope == target_scope &&
1285             agent_live_query_key_equal(source_key, target_key))
1286             attributed = source;

```



```

1287     intr_restore(enabled);
1288     for (uint stream = 0; stream < 2; stream++) {
1289         uint changes = stream == 0 ? legacy_changes : typed_changes;
1290         int event_type = stream == 0 ? AGENT_EVENT_FILE_STATUS :
1291             AGENT_EVENT_FILE_QUERY;
1292
1293         for (int change = AGENT_LIVE_QUERY_ENTER;
1294             change <= AGENT_LIVE_QUERY_LEAVE; change++) {
1295             char payload[AGENT_EVENT_PAYLOAD_SIZE];
1296             int result;
1297
1298             if ((changes & (1U << change)) == 0)
1299                 continue;
1300             agent_live_query_payload(payload, change, key);
1301             result = agent_ipc_deliver_live_event(
1302                 target, attributed, event_type, fid,
1303                 generation, payload, 0);
1304             if (result > 0) {
1305                 delivered += result;
1306                 continue;
1307             }
1308             if (result < 0) {
1309                 enabled = intr_save();
1310                 agent_live_query_proc_resync_mark_locked(
1311                     target, generation,
1312                     1U << event_type);
1313                 intr_restore(enabled);
1314                 (void)agent_live_query_proc_resync_flush(
1315                     target);
1316             }
1317         }
1318     }
1319 }
1320 return delivered;
1321 }

```

代码 15: 实时查询的重新同步确认与 workflow 屏障入口

c

```

1323 void
1324 agent_live_query_watch_installed(struct proc *p)
1325 {
1326     struct workflow_lifecycle_key key;
1327     uint64 generation;
1328     uint scope_id;
1329     int enabled = intr_save();
1330
1331     if (!agent_live_query_target_valid(p, &key, &scope_id) ||
1332         !agent_live_query_has_file_watch(p))
1333         goto out;
1334     generation = agent_file_state_scope_generation(scope_id);
1335     generation = MAX(generation,
1336         agent_live_query_domain_generation_locked(key, scope_id));
1337     agent_live_query_proc_resync_mark_locked(
1338         p, generation, 1U << AGENT_EVENT_FILE_STATUS);
1339     (void)agent_live_query_proc_resync_flush(p);
1340 out:

```

```

1341     intr_restore(enabled);
1342 }
1343
1344 void
1345 agent_live_query_watch_removed(struct proc *p)
1346 {
1347     int enabled;
1348     uint slot;
1349
1350     if (p == 0 || p < pool || p >= &pool[NPROC])
1351         return;
1352     enabled = intr_save();
1353     if (!agent_live_query_has_file_watch(p)) {
1354         slot = p - pool;
1355         agent_live_query_proc_resync_clear(
1356             &agent_live_query_proc_resync[slot]);
1357     }
1358     intr_restore(enabled);
1359 }
1360
1361 void
1362 agent_live_query_proc_reset(struct proc *p)
1363 {
1364     int enabled;
1365
1366     if (p == 0 || p < pool || p >= &pool[NPROC])
1367         return;
1368     enabled = intr_save();
1369     for (uint slot = 0; slot < AGENT_LIVE_QUERY_SUBSCRIPTION_CAP; slot++)
1370         if (agent_live_query_subscriptions[slot].used &&
1371             agent_live_query_subscriptions[slot].target == p)
1372             agent_live_query_subscription_remove_locked(
1373                 &agent_live_query_subscriptions[slot], 0);
1374     agent_live_query_proc_resync_clear(
1375         &agent_live_query_proc_resync[p - pool]);
1376     intr_restore(enabled);
1377 }
1378
1379 int
1380 agent_live_query_resync_ack(struct workflow_lifecycle_key key,
1381     uint scope_id, uint64 generation)
1382 {
1383     int acknowledged = 0;
1384     int enabled;
1385
1386     if (!agent_live_query_domain_valid(key, scope_id) || generation == 0)
1387         return 0;
1388     enabled = intr_save();
1389     for (uint slot = 0; slot < AGENT_LIVE_QUERY_DOMAIN_CAP; slot++) {
1390         struct agent_live_query_domain_resync *state =
1391             &agent_live_query_domain_resync[slot];
1392
1393         if (!state->used || state->generation > generation ||
1394             !agent_live_query_domain_equal(
1395                 state->key, state->scope_id, key, scope_id))
1396             continue;
1397         memset(state, 0, sizeof(*state));

```

```

1398     acknowledged = 1;
1399 }
1400 if (scope_id == VFS_SCOPE_SYSTEM &&
1401     agent_live_query_global_resync_generation != 0 &&
1402     agent_live_query_global_resync_generation <= generation) {
1403     agent_live_query_global_resync_generation = 0;
1404     acknowledged = 1;
1405 }
1406 intr_restore(enabled);
1407 return acknowledged;
1408 }
1409
1410 static int
1411 agent_live_query_proc_resync_pending_domain(
1412     struct workflow_lifecycle_key key, uint scope_id)
1413 {
1414     for (struct proc *target = pool; target < &pool[NPROC]; target++) {
1415         struct agent_live_query_proc_resync *pending =
1416             &agent_live_query_proc_resync[target - pool];
1417
1418         if (!agent_live_query_proc_resync_valid(pending, target))
1419             continue;
1420         if (scope_id == VFS_SCOPE_SYSTEM ||
1421             agent_live_query_domain_equal(
1422                 pending->key, pending->scope_id, key, scope_id))
1423             return 1;
1424     }
1425     return 0;
1426 }
1427
1428 int
1429 agent_live_query_fence_drain(struct workflow_lifecycle_key key, uint scope_id)
1430 {
1431     int retry = 0;
1432
1433     if (!agent_live_query_domain_valid(key, scope_id) ||
1434         !agent_metadata_txn_owned(0))
1435         return AGENT_STATUS_RETRY;
1436     for (uint slot = 0; slot < AGENT_LIVE_QUERY_TOMBSTONE_CAP; slot++) {
1437         int pending;
1438         int enabled = intr_save();
1439
1440         pending = agent_live_query_tombstones[slot].used &&
1441             agent_live_query_domain_equal(
1442                 agent_live_query_tombstones[slot].key,
1443                 agent_live_query_tombstones[slot].scope_id,
1444                 key, scope_id);
1445         intr_restore(enabled);
1446         if (!pending)
1447             continue;
1448         if (agent_live_query_tombstone_process(slot) < 0)
1449             retry = 1;
1450     }
1451     for (uint slot = 0; slot < AGENT_FILE_META_MAX; slot++) {
1452         int pending;
1453         int enabled = intr_save();
1454     }

```

```

1455     pending = agent_live_query_content_pending[slot].used &&
1456         agent_live_query_domain_equal(
1457             agent_live_query_content_pending[slot]
1458                 .receipt.lifecycle,
1459             agent_live_query_content_pending[slot]
1460                 .receipt.scope_id,
1461             key, scope_id);
1462     intr_restore(enabled);
1463     if (!pending)
1464         continue;
1465     if (agent_live_query_content_process(slot) < 0)
1466         retry = 1;
1467 }
1468 for (struct proc *target = pool; target < &pool[NPROC]; target++) {

```

3.8.5. 任务通道与等待路径

任务通道设置只允许智能体主线程使用；SQ 和 CQ 各占一个客户机映射页，复制后的请求和完成记录，以及权威资源状态分别放在两页内核私有页。进入操作和资源绑定会记录发起线程编号和身份代次。进入内核后，代码仍检查进入 ABI、CQ 确认、重新同步标志和 SQ 边界。发起者、创建者或生命周期不匹配导致的 STALE 只保留 ABI 版本、大小和 status，其他输出为零；控制代次不匹配时，结果填充函数返回当前通道状态。CQ_FULL 只是完成队列背压，不会把已接纳的请求直接变为完成记录。目标已确认或不存在时，取消操作仍会消费取消编号，结果填充函数返回当前代次和水位线，也不会额外生成取消 CQE。环形队列或槽位协议错误进入黏性重新同步后，发起者必须以空 SQ 尾指针的显式重新同步进入操作复位。这里的队列是受执行约定和执行记录约束的内核对象，不是普通用户态环形数组。

代码 16：任务通道进入操作的代次、重新同步与 SQ 边界检查

c

```

1937 int
1938 agent_task_channel_enter(struct proc *p, struct thread *issuer,
1939     const struct agent_task_channel_enter *enter,
1940     struct agent_task_channel_enter_result *result,
1941     const struct agent_task_channel_ops *ops)
1942 {
1943     struct agent_task_channel_state *state;
1944     struct agent_task_private_header *header;
1945     uint submitted = 0;
1946     uint completed = 0;
1947     uint limit;
1948     int status = AGENT_TASK_CHANNEL_OK;
1949     int enabled;
1950
1951     if (result == 0)
1952         return AGENT_TASK_CHANNEL_BAD_REQUEST;
1953     memset(result, 0, sizeof(*result));
1954     result->version = AGENT_TASK_CHANNEL_VERSION;
1955     result->size = sizeof(*result);
1956     result->status = AGENT_TASK_CHANNEL_BAD_REQUEST;
1957     if (p == 0 || issuer == 0 || enter == 0 ||
1958         enter->version != AGENT_TASK_CHANNEL_VERSION ||

```

```

1959     enter->size != sizeof(*enter) ||
1960     (enter->flags & ~AGENT_TASK_CHANNEL_ENTER_F_ALL) != 0 ||
1961     ((enter->flags & AGENT_TASK_CHANNEL_ENTER_F_DRAIN) != 0 &&
1962     enter->max_submit != 0) ||
1963     enter->max_submit > AGENT_TASK_CHANNEL_CAPACITY ||
1964     enter->min_complete > AGENT_TASK_CHANNEL_CAPACITY ||
1965     enter->reserved != 0 || enter->reserved_tail[0] != 0 ||
1966     enter->reserved_tail[1] != 0)
1967     return result->status;
1968     limit = (enter->flags & AGENT_TASK_CHANNEL_ENTER_F_DRAIN) != 0 ? 0 :
1969     (enter->max_submit == 0 ? AGENT_TASK_CHANNEL_CAPACITY :
1970     enter->max_submit);
1971     enabled = intr_save();
1972     state = agent_task_channel_find_locked(p);
1973     if (!agent_task_channel_issuer_valid_locked(state, p, issuer) ||
1974     state->ops != ops) {
1975         intr_restore(enabled);
1976         result->status = AGENT_TASK_CHANNEL_STALE;
1977         return result->status;
1978     }
1979     header = &state->private->header;
1980     if (enter->generation != header->generation) {
1981         agent_task_channel_enter_result_fill(
1982             state, result, AGENT_TASK_CHANNEL_STALE, 0, 0);
1983         intr_restore(enabled);
1984         return result->status;
1985     }
1986     status = agent_task_channel_ack_locked(state, enter->cq_head);
1987     if (status < 0) {
1988         agent_task_channel_enter_result_fill(state, result, status, 0, 0);
1989         intr_restore(enabled);
1990         return result->status;
1991     }
1992     header = &state->private->header;
1993     if ((header->flags & AGENT_TASK_CHANNEL_RING_F_RESYNC) != 0) {
1994         if ((enter->flags & AGENT_TASK_CHANNEL_ENTER_F_RESYNC) == 0 ||
1995         enter->sq_tail != 0) {
1996             agent_task_channel_enter_result_fill(
1997                 state, result,
1998                 AGENT_TASK_CHANNEL_RESYNC_REQUIRED, 0, 0);
1999             intr_restore(enabled);
2000             return result->status;
2001         }
2002         header->flags &= ~AGENT_TASK_CHANNEL_RING_F_RESYNC;
2003     } else if ((enter->flags & AGENT_TASK_CHANNEL_ENTER_F_RESYNC) != 0) {
2004         agent_task_channel_enter_result_fill(
2005             state, result, AGENT_TASK_CHANNEL_BAD_REQUEST, 0, 0);
2006         intr_restore(enabled);
2007         return result->status;
2008     }
2009     if (enter->sq_tail < header->sq_tail ||
2010     enter->sq_tail < header->sq_head ||
2011     enter->sq_tail - header->sq_head > AGENT_TASK_CHANNEL_CAPACITY) {
2012         status = agent_task_channel_protocol_fault_locked(state);
2013         agent_task_channel_enter_result_fill(state, result, status, 0, 0);
2014         intr_restore(enabled);
2015         return result->status;

```

```

2016     }
2017     header->sq_tail = enter->sq_tail;
2018     completed += agent_task_channel_flush_locked(state);
2019     agent_task_channel_publish_locked(state);
2020     intr_restore(enabled);

```

任务的资源导入接口明确拒绝任何请求，已占用资源不能作为隐式预算转入任务。这样，接纳检查仍能依据执行约定声明的资源额度作出可预测的决定；复杂的资源移交必须通过明确的工作流资源接口完成。

3.8.6. 资源使用租约与工作流 EEVDF

资源控制器通过资源使用租约锁定并激活一组资源账户额度。租约记录创建者、生命周期、代次和账户对；激活时，内核将租约令牌写入线程冷状态，停用、结算和放弃时都必须匹配同一代次。这样，一段工具执行只能使用受限且可结算的资源额度。

代码 17：资源使用租约的创建与账户对提交

c

```

1770 int resource_phase_lease_begin(
1771     struct resource_phase_lease *lease,
1772     struct workflow_lifecycle_key lifecycle,
1773     struct resource_account_handle exec_handle,
1774     struct resource_account_handle storage_handle,
1775     enum resource_charge_class charge_class,
1776     const uint64 exec_amounts[RESOURCE_KIND_COUNT],
1777     const uint64 storage_amounts[RESOURCE_KIND_COUNT], uint node_id,
1778     uint64 request_id)
1779 {
1780     struct resource_account *accounts[RESOURCE_PHASE_ACCOUNT_COUNT];
1781     struct resource_phase_account_lock
1782         *locks[RESOURCE_PHASE_ACCOUNT_COUNT];
1783     struct resource_account_handle handles[RESOURCE_PHASE_ACCOUNT_COUNT] = {
1784         [RESOURCE_PHASE_EXEC] = exec_handle,
1785         [RESOURCE_PHASE_STORAGE] = storage_handle,
1786     };
1787     uint64 amounts[RESOURCE_PHASE_ACCOUNT_COUNT][RESOURCE_KIND_COUNT];
1788     uint masks[RESOURCE_PHASE_ACCOUNT_COUNT], union_mask;
1789     struct resource_phase_registry_entry *entry;
1790     struct thread *owner;
1791     int registry_slot;
1792     int enabled;
1793
1794     if (lease == 0 || lease->state != RESOURCE_PHASE_LEASE_EMPTY ||
1795         exec_amounts == 0 || storage_amounts == 0 || request_id == 0 ||
1796         !workflow_lifecycle_key_valid(lifecycle) ||
1797         !resource_charge_class_valid(charge_class) ||
1798         resource_account_handle_equal(exec_handle, storage_handle))
1799         return -1;
1800     memmove(amounts[RESOURCE_PHASE_EXEC], exec_amounts,
1801             sizeof(amounts[RESOURCE_PHASE_EXEC]));
1802     memmove(amounts[RESOURCE_PHASE_STORAGE], storage_amounts,
1803             sizeof(amounts[RESOURCE_PHASE_STORAGE]));
1804     for (uint role = 0; role < RESOURCE_PHASE_ACCOUNT_COUNT; role++) {
1805         for (uint kind = 0; kind < RESOURCE_KIND_COUNT; kind++)

```



```

1806         if (amounts[role][kind] > (uint)-1)
1807             return -1;
1808         masks[role] = resource_phase_amount_mask(amounts[role]);
1809     }
1810     union_mask = masks[RESOURCE_PHASE_EXEC] |
1811                 masks[RESOURCE_PHASE_STORAGE];
1812     if (union_mask == 0)
1813         return -1;
1814
1815     enabled = intr_save();
1816     owner = curr_thread();
1817     registry_slot = resource_phase_registry_free_slot();
1818     accounts[RESOURCE_PHASE_EXEC] = resource_account_lookup(exec_handle);
1819     accounts[RESOURCE_PHASE_STORAGE] =
1820         resource_account_lookup(storage_handle);
1821     if (accounts[RESOURCE_PHASE_EXEC] == 0 ||
1822         accounts[RESOURCE_PHASE_STORAGE] == 0 ||
1823         accounts[RESOURCE_PHASE_EXEC]->kind != RESOURCE_ACCOUNT_EXEC ||
1824         accounts[RESOURCE_PHASE_STORAGE]->kind !=
1825             RESOURCE_ACCOUNT_STORAGE ||
1826         registry_slot < 0 || owner == 0 || owner->trapframe == 0 ||
1827         owner->identity_generation == 0 || owner->process == 0 ||
1828         !proc_teartdown_live(owner->process) ||
1829         resource_phase_registry_owner_busy(owner) ||
1830         charge_class !=
1831             (owner->process->resource_slot_reserved ?
1832              RESOURCE_CHARGE_RESERVED :
1833              RESOURCE_CHARGE_ORDINARY) ||
1834         resource_phase_generation == RESOURCE_LIMIT_UNBOUNDED ||
1835         !workflow_lifecycle_active(lifecycle) ||
1836         !resource_phase_owner_binding_matches(
1837             lifecycle, exec_handle, owner) ||
1838         !resource_phase_lifecycle_pair_admissible(
1839             lifecycle, handles) ||
1840         !resource_phase_lock_records_locked(accounts, handles, locks) ||
1841         !resource_phase_pair_admissible_locked(
1842             accounts, charge_class, amounts, masks, union_mask)) {
1843         intr_restore(enabled);
1844         return -1;
1845     }
1846     resource_phase_pair_reclaim_locked(
1847         accounts, charge_class, amounts, union_mask);
1848     resource_phase_pair_commit_locked(
1849         accounts, handles, locks, charge_class, amounts, masks);
1850     resource_phase_generation++;
1851     entry = &resource_phase_registry[registry_slot];
1852     memset(entry, 0, sizeof(*entry));
1853     entry->owner = owner;
1854     entry->lease.lifecycle = lifecycle;
1855     entry->lease.generation = resource_phase_generation;
1856     entry->lease.request_id = request_id;
1857     entry->lease.owner_thread_generation = owner->identity_generation;
1858     entry->lease.node_id = node_id;
1859     entry->lease.charge_class = charge_class;
1860     entry->lease.state = RESOURCE_PHASE_LEASE_ADMITTED;
1861     for (uint role = 0; role < RESOURCE_PHASE_ACCOUNT_COUNT; role++) {
1862         entry->lease.account[role].handle = handles[role];

```

```

1863     entry->lease.account[role].kind_mask = masks[role];
1864     for (uint kind = 0; kind < RESOURCE_KIND_COUNT; kind++)
1865         entry->lease.account[role].remaining[kind] =
1866             (uint) amounts[role][kind];
1867     }
1868     entry->used = 1;
1869     lease->generation = resource_phase_generation;
1870     lease->request_id = request_id;
1871     lease->node_id = node_id;
1872     lease->registry_slot = (uint) registry_slot;
1873     lease->state = RESOURCE_PHASE_LEASE_ADMITTED;
1874     intr_restore(enabled);
1875     return 0;
1876 }

```

代码 18: 资源使用租约的线程激活与去激活

c

```

1878 int resource_phase_lease_activate(struct resource_phase_lease *lease,
1879     uint64 expected_generation)
1880 {
1881     struct thread *thread;
1882     struct resource_account *account;
1883     struct resource_phase_registry_entry *entry;
1884     struct resource_phase_lease_record *record;
1885     int enabled;
1886
1887     if (lease == 0 || expected_generation == 0)
1888         return -1;
1889     enabled = intr_save();
1890     thread = curr_thread();
1891     entry = resource_phase_registry_lookup(
1892         lease, expected_generation);
1893     record = entry == 0 ? 0 : &entry->lease;
1894     if (entry == 0 || record->state != RESOURCE_PHASE_LEASE_ADMITTED ||
1895         thread == 0 || entry->owner != thread ||
1896         record->owner_thread_generation != thread->identity_generation ||
1897         thread->trapframe == 0 ||
1898         thread->identity_generation == 0 ||
1899         thread_trap_cold(thread)->resource_phase_lease_generation != 0 ||
1900         thread_trap_cold(thread)->resource_phase_claim_depth != 0 ||
1901         !proc_teardown_live(thread->process) ||
1902         !workflow_lifecycle_active(record->lifecycle) ||
1903         !resource_phase_owner_matches(record, thread))
1904         goto fail;
1905     for (uint role = 0; role < RESOURCE_PHASE_ACCOUNT_COUNT; role++) {
1906         account = resource_account_lookup(record->account[role].handle);
1907         if (account == 0 || account->state != RESOURCE_ACCOUNT_ACTIVE ||
1908             resource_phase_account_lock_lookup(
1909                 record->account[role].handle) == 0)
1910             goto fail;
1911     }
1912     record->owner_thread_generation = thread->identity_generation;
1913     record->state = RESOURCE_PHASE_LEASE_ACTIVE;
1914     lease->state = RESOURCE_PHASE_LEASE_ACTIVE;
1915     thread_trap_cold(thread)->resource_phase_lease_slot =

```

```

1916     lease->registry_slot;
1917     thread_trap_cold(thread)->resource_phase_lease_generation =
1918         expected_generation;
1919     intr_restore(enabled);
1920     return 0;
1921 fail:
1922     intr_restore(enabled);
1923     return -1;
1924 }
1925
1926 int resource_phase_lease_deactivate(struct resource_phase_lease *lease,
1927     uint64 expected_generation)
1928 {
1929     struct thread *thread;
1930     struct resource_phase_registry_entry *entry;
1931     struct resource_phase_lease_record *record;
1932     int enabled;
1933
1934     if (lease == 0 || expected_generation == 0)
1935         return -1;
1936     enabled = intr_save();
1937     thread = curr_thread();
1938     entry = resource_phase_registry_lookup(
1939         lease, expected_generation);
1940     record = entry == 0 ? 0 : &entry->lease;
1941     if (entry == 0 || record->state != RESOURCE_PHASE_LEASE_ACTIVE ||
1942         thread == 0 || entry->owner != thread ||
1943         thread->trapframe == 0 ||
1944         thread_trap_cold(thread)->resource_phase_lease_slot !=
1945             lease->registry_slot ||
1946         thread_trap_cold(thread)->resource_phase_lease_generation !=
1947             expected_generation ||
1948         record->owner_thread_generation != thread->identity_generation ||
1949         record->outstanding_claims != 0 ||
1950         thread_trap_cold(thread)->resource_phase_claim_depth != 0) {
1951         intr_restore(enabled);
1952         return -1;
1953     }
1954     thread_trap_cold(thread)->resource_phase_lease_slot =
1955         RESOURCE_PHASE_REGISTRY_SLOT_NONE;
1956     thread_trap_cold(thread)->resource_phase_lease_generation = 0;
1957     record->state = RESOURCE_PHASE_LEASE_DEACTIVATED;
1958     lease->state = RESOURCE_PHASE_LEASE_DEACTIVATED;
1959     intr_restore(enabled);
1960     return 0;
1961 }

```

调度器不会因为同一工作流拥有更多线程，就把它当作更多独立的公平实体。workflow_scheduler_select() 先收集可运行工作流，再依据滞后量、截止时间、权重和可运行负载选择实体；调度和记账分别记录服务开始时刻与已消耗时间。因此，调度公平性按工作流记账，而不是按偶然增加的线程数记账。

代码 19: workflow EEVDF 的候选收集与选择

c

```

408 int workflow_scheduler_select(
409     const struct workflow_scheduler_candidate *candidates, uint count,
410     uint64 now_cycle, uint64 now_tick, int *selected_domain)
411 {
412     int slots[WORKFLOW_SCHEDULER_ENTITY_CAP];
413     uint planned_latency[WORKFLOW_SCHEDULER_ENTITY_CAP];
414     uint planned_request_ticks[WORKFLOW_SCHEDULER_ENTITY_CAP];
415     uint planned_decays[WORKFLOW_SCHEDULER_ENTITY_CAP];
416     uint64 planned_vruntime[WORKFLOW_SCHEDULER_ENTITY_CAP];
417     uint64 planned_remaining[WORKFLOW_SCHEDULER_ENTITY_CAP];
418     uint64 planned_deadline[WORKFLOW_SCHEDULER_ENTITY_CAP];
419     uint reserved = 0;
420     uint new_slots = 0;
421     uint wake_transitions = 0;
422     uint64 rebase_offset;
423     uint64 planned_vtime;
424     int selected = -1;
425     int enabled;
426
427     (void)now_cycle;
428     if (selected_domain == 0 || candidates == 0 || count == 0 ||
429         count > WORKFLOW_SCHEDULER_ENTITY_CAP)
430         return -1;
431     *selected_domain = -1;
432     for (uint i = 0; i < WORKFLOW_SCHEDULER_ENTITY_CAP; i++)
433         slots[i] = WORKFLOW_SCHEDULER_SLOT_NONE;
434     enabled = intr_save();
435     /* Phase one is read-only: validate every identity and reserve local slots. */
436     for (uint i = 0; i < count; i++) {
437         int is_new = 0;
438
439         if (!workflow_scheduler_identity_valid(
440             candidates[i].lifecycle, candidates[i].account,
441             candidates[i].domain_id) ||
442             candidates[i].runnable_threads == 0 ||
443             candidates[i].runnable_threads > 0xffffU ||
444             candidates[i].latency_class >
445             AGENT_WORKFLOW_LATENCY_BATCH)
446             goto fail;
447         for (uint j = 0; j < i; j++)
448             if (candidates[j].domain_id ==
449                 candidates[i].domain_id ||
450                 workflow_lifecycle_key_equal(
451                     candidates[j].lifecycle,
452                     candidates[i].lifecycle) ||
453                 resource_account_handle_equal(
454                     candidates[j].account,
455                     candidates[i].account))
456                 goto fail;
457         planned_request_ticks[i] = workflow_scheduler_request_ticks(
458             candidates[i].latency_class,
459             candidates[i].deadline_delta_ticks);
460         if (planned_request_ticks[i] == 0)
461             goto fail;

```

```

462     slots[i] = workflow_scheduler_plan_slot_locked(
463         &candidates[i], reserved, &is_new);
464     if (slots[i] < 0)
465         goto fail;
466     reserved |= 1U << slots[i];
467     if (is_new)
468         new_slots |= 1U << i;
469     else if (workflow_scheduler_entities[slots[i]].flags &
470         WORKFLOW_SCHEDULER_F_CACHE_INVALID)
471         goto fail;
472 }
473
474 /* Compute the entire post-selection state locally before any mutation. */
475 rebase_offset = workflow_scheduler_rebase_offset_locked();
476 planned_vtime = workflow_scheduler_vtime >= rebase_offset ?
477     workflow_scheduler_vtime - rebase_offset : 0;
478 for (uint i = 0; i < count; i++) {
479     struct workflow_scheduler_entity *entity =
480         &workflow_scheduler_entities[slots[i]];
481     uint64 request_cycles = workflow_scheduler_request_cycles(
482         planned_request_ticks[i]);
483
484     planned_decays[i] = 0;
485     if (new_slots & (1U << i)) {
486         planned_vruntime[i] = planned_vtime;
487         planned_remaining[i] = 0;
488         planned_latency[i] = AGENT_WORKFLOW_LATENCY_NORMAL;
489         wake_transitions |= 1U << i;
490     } else {
491         planned_vruntime[i] = entity->vruntime - rebase_offset;
492         planned_remaining[i] = entity->remaining_cycles;
493         planned_latency[i] = entity->latency_class;
494         planned_request_ticks[i] = entity->request_ticks;
495         if (!(entity->flags &
496             AGENT_WORKFLOW_SCHED_F_RUNNABLE)) {
497             wake_transitions |= 1U << i;
498             planned_remaining[i] = 0;
499             if (entity->flags &
500                 AGENT_WORKFLOW_SCHED_F_SLEEPING) {
501                 uint64 slept =
502                     now_tick >= entity->sleep_start_tick ?
503                     now_tick -
504                     entity->sleep_start_tick :
505                     0;
506
507                 planned_decays[i] = slept /
508                     WORKFLOW_SCHEDULER_SLEEP_DECAY_TICKS;
509                 if (planned_decays[i] >
510                     WORKFLOW_SCHEDULER_MAX_SLEEP_DECAYS)
511                     planned_decays[i] =
512                         WORKFLOW_SCHEDULER_MAX_SLEEP_DECAYS;
513                 planned_vruntime[i] =
514                     workflow_scheduler_decay_toward(
515                         planned_vruntime[i],
516                         planned_vtime,
517                         planned_decays[i]);
518             }

```

```

519     }
520 }
521 /* Urgency may shorten an open request, never mint more service. */
522 if (planned_remaining[i] == 0 ||
523     request_cycles < planned_remaining[i]) {
524     planned_remaining[i] = request_cycles;
525     planned_request_ticks[i] = workflow_scheduler_request_ticks(
526         candidates[i].latency_class,
527         candidates[i].deadline_delta_ticks);
528     planned_latency[i] = candidates[i].latency_class;
529 }
530 if (planned_vruntime[i] > ~0ULL - planned_remaining[i])
531     goto fail;
532 planned_deadline[i] =
533     planned_vruntime[i] + planned_remaining[i];
534 }
535 if (count == 1) {
536     if (planned_vruntime[0] > planned_vtime)
537         planned_vtime = planned_vruntime[0];
538 } else {
539     uint64 average = workflow_scheduler_average(
540         planned_vruntime, count);
541
542     if (average > planned_vtime)
543         planned_vtime = average;
544 }
545 for (uint i = 0; i < count; i++) {
546     if (planned_vruntime[i] <= planned_vtime) {
547         if (selected < 0 ||
548             workflow_scheduler_before(candidates,
549                                     planned_deadline,
550                                     planned_vruntime, i,
551                                     (uint)selected))
552             selected = i;
553     }
554 }
555 if (selected < 0)
556     goto fail;

```

代码 20: workflow EEVDF 的提交、调度与记账

c

```

558 /* Phase two commits a plan that can no longer fail. */
559 if (rebase_offset != 0)
560     workflow_scheduler_rebase_locked();
561 for (uint i = 0; i < count; i++) {
562     struct workflow_scheduler_entity *entity =
563         &workflow_scheduler_entities[slots[i]];
564
565     if (new_slots & (1U << i)) {
566         memset(entity, 0, sizeof(*entity));
567         entity->lifecycle = candidates[i].lifecycle;
568         entity->account = candidates[i].account;
569         entity->domain_id = candidates[i].domain_id;
570         entity->flags = AGENT_WORKFLOW_SCHED_F_ACTIVE;
571         entity->earliest_deadline_tick =
572             WORKFLOW_SCHEDULER_NO_DEADLINE;

```

```

573     entity->cached_latency_class =
574         (uchar)candidates[i].latency_class;
575     entity->schedulable_threads =
576         (ushort)candidates[i].runnable_threads;
577 }
578 workflow_scheduler_domain_slot[candidates[i].domain_id] =
579     (signed char)slots[i];
580 entity->vruntime = planned_vruntime[i];
581 entity->remaining_cycles = planned_remaining[i];
582 entity->virtual_deadline = planned_deadline[i];
583 entity->request_ticks = (uchar)planned_request_ticks[i];
584 entity->latency_class = (uchar)planned_latency[i];
585 entity->runnable_threads =
586     (ushort)candidates[i].runnable_threads;
587 entity->mode = AGENT_WORKFLOW_SCHED_MODE_EEVDF;
588 if (wake_transitions & (1U << i)) {
589     entity->sleep_decays = workflow_scheduler_counter32_add(
590         entity->sleep_decays, planned_decays[i]);
591     entity->wake_tick = now_tick;
592     entity->flags |= WORKFLOW_SCHEDULER_F_WAKE_PENDING;
593 }
594 entity->flags &= ~(AGENT_WORKFLOW_SCHED_F_SLEEPING |
595     AGENT_WORKFLOW_SCHED_F_ELIGIBLE |
596     AGENT_WORKFLOW_SCHED_F_FALLBACK);
597 entity->flags |= AGENT_WORKFLOW_SCHED_F_RUNNABLE;
598 if (planned_vruntime[i] <= planned_vtime)
599     entity->flags |= AGENT_WORKFLOW_SCHED_F_ELIGIBLE;
600 else
601     entity->eligibility_misses =
602         workflow_scheduler_counter32_add(
603             entity->eligibility_misses, 1);
604 }
605 workflow_scheduler_vtime = planned_vtime;
606 if (candidates[selected].deadline_delta_ticks == 0) {
607     struct workflow_scheduler_entity *entity =
608         &workflow_scheduler_entities[slots[selected]];
609
610     if (!(entity->flags & WORKFLOW_SCHEDULER_F_DEADLINE_MISSED)) {
611         entity->flags |= WORKFLOW_SCHEDULER_F_DEADLINE_MISSED;
612         entity->deadline_misses = workflow_scheduler_counter32_add(
613             entity->deadline_misses, 1);
614     }
615 }
616 *selected_domain = candidates[selected].domain_id;
617 intr_restore(enabled);
618 return 0;

```

算法 4：资源与 CPU 的工作流级执行过程

1. 执行约定把节点的执行额度、存储额度和记账类别写入当前声明。
2. 资源使用租约锁定、激活并结算本阶段的账户额度；超限或状态失效时返回受控结果。
3. 可运行线程按工作流汇集，EEVDF 根据实体的滞后量和截止时间选择下一次服务。
4. 执行约定路径在上下文和执行记录中写入完成状态，使资源拒绝、取消和超时能够

回溯到具体节点和尝试号。

3.8.7. 客户机运行时与结果文件发布

Nexus 用户态库会重新发现内核工具目录，逐项核对 ABI、编号、名称和参数规则是否一致；发起调用时还会检查当前智能体身份、产品角色和必要能力位。用户态校验不替代内核，而是让模型适配层尽早排除不可能成功的工具调用。

代码 21: Nexus 对内核工具目录的发现与一致性检查

c

```

681 int agent_nexus_tools_discover(void)
682 {
683     unsigned char seen[AGENT_TOOL_COUNT];
684     int count;
685
686     memset(nexus_tool_catalog, 0, sizeof(nexus_tool_catalog));
687     memset(seen, 0, sizeof(seen));
688     nexus_tool_catalog_count = 0;
689     count = tool_list(nexus_tool_catalog, AGENT_TOOL_COUNT);
690     if (count != AGENT_TOOL_COUNT)
691         return -1;
692     for (int i = 0; i < count; i++) {
693         const struct agent_tool_desc_v2 *descriptor =
694             &nexus_tool_catalog[i];
695         const struct agent_nexus_tool_spec *spec;
696
697         if (descriptor->version != AGENT_CALL_VERSION_V2 ||
698             nexus_tool_catalog[i].size != sizeof(nexus_tool_catalog[i]) ||
699             descriptor->tool_id <= 0 ||
700             descriptor->tool_id > AGENT_TOOL_COUNT ||
701             strlen(descriptor->name, sizeof(descriptor->name)) ==
702                 sizeof(descriptor->name) ||
703             strlen(descriptor->params, sizeof(descriptor->params)) ==
704                 sizeof(descriptor->params) ||
705             seen[descriptor->tool_id - 1] != 0)
706             goto invalid;
707         spec = &nexus_tool_specs[descriptor->tool_id - 1];
708         if (spec->tool_id != descriptor->tool_id ||
709             !nexus_text_equal_bounded(spec->name, descriptor->name,
710                                     AGENT_TOOL_NAME_SIZE) ||
711             !nexus_text_equal_bounded(spec->parameters,
712                                     descriptor->params,
713                                     AGENT_TOOL_PARAMS_SIZE))
714             goto invalid;
715         seen[descriptor->tool_id - 1] = 1;
716     }
717     nexus_tool_catalog_count = count;
718     return count;
719
720 invalid:
721     memset(nexus_tool_catalog, 0, sizeof(nexus_tool_catalog));
722     return -1;
723 }

```

代码 22: Nexus 带类型信息工具调用的用户态封装

c

```

894 int agent_nexus_tool_call(const char *name, unsigned long long request_id,
895                          const struct agent_nexus_tool_argument *arguments,
896                          unsigned int argument_count,
897                          struct agent_response_v2 *response)
898 {
899     const struct agent_tool_desc_v2 *tool = agent_nexus_tool_find(name);
900     const struct agent_nexus_tool_spec *spec = agent_nexus_tool_spec_find(name);
901     struct agent_request_v2 request;
902     struct agent_nexus_artifact_actor identity;
903     struct agent_info info;
904     unsigned int role_bit;
905
906     if (tool == 0 || spec == 0 || response == 0 || request_id == 0 ||
907         argument_count > AGENT_TOOL_PARAM_MAX ||
908         (argument_count != 0 && arguments == 0) ||
909         (tool->flags & AGENT_TOOL_F_CALLABLE) == 0 ||
910         argument_count > tool->param_count ||
911         !nexus_schema_arguments_valid(spec, arguments, argument_count))
912         return -1;
913     if (agent_nexus_identity_current(&identity) < 0)
914         return -1;
915     role_bit = AGENT_NEXUS_TOOL_ROLE(identity.product_role);
916     memset(&info, 0, sizeof(info));
917     if (agent_info(&info) != 0 || !info.is_agent ||
918         info.agent_id != (int)identity.agent_id ||
919         (spec->product_role_mask & role_bit) == 0 ||
920         (info.capability_mask & spec->required_capabilities) !=
921         spec->required_capabilities)
922         return -1;
923     memset(&request, 0, sizeof(request));
924     memset(response, 0, sizeof(*response));
925     memset(nexus_tool_params, 0, sizeof(nexus_tool_params));
926     request.version = AGENT_CALL_VERSION_V2;
927     request.size = sizeof(request);
928     request.tool_id = tool->tool_id;
929     request.param_count = argument_count;
930     request.request_id = request_id;
931     request.params = argument_count ?
932         (unsigned long long)nexus_tool_params : 0;
933     strcpy(request.tool_name, tool->name);
934     for (unsigned int i = 0; i < argument_count; i++) {
935         struct agent_param_v2 *param = &nexus_tool_params[i];
936
937         param->version = AGENT_PARAM_VERSION;
938         param->size = sizeof(*param);
939         param->type = arguments[i].type;
940         strcpy(param->key, arguments[i].key);
941         if (arguments[i].type == AGENT_PARAM_UINT64) {
942             param->value_size = sizeof(unsigned long long);
943             param->value.uint64_value = arguments[i].number;
944         } else {
945             unsigned int length = strlen(arguments[i].text,
946             sizeof(arguments[i].text)) + 1U;
947

```

```

948     param->value_size = length;
949     strcpy(param->value.string_value, arguments[i].text);
950 }
951 }
952 if (tool_call(&request, response) != 0 ||
953     response->version != AGENT_CALL_VERSION_V2 ||
954     response->size != sizeof(*response) ||
955     response->request_id != request_id || response->tool_id != tool->tool_id ||
956     strlen(response->tool_name, sizeof(response->tool_name)) ==
957     sizeof(response->tool_name) ||
958     !nexus_text_equal_bounded(response->tool_name, tool->name,
959                             AGENT_TOOL_NAME_SIZE))
960     return -1;
961 return 0;

```

结果文件发布使用 workflow 访问范围内的编辑租约限制并发布者。发布者先写入带无效魔数的待完成头部、数据内容并执行 `fsync`，最后才写入有效头部。`uCore` 没有原子的链接或重命名提交，因此该 **ABI** 不是原子发布：崩溃恢复时会留下不可读的 `publish-once` 墓碑记录，而不是可读的半成品。

编辑租约的提交或放弃允许当前创建者操作，也允许与该租约同访问范围、同生命周期且持有 `ORCHESTRATE` 的智能体操作。到期清理后找不到该租约时返回 `NOT_FOUND`，实例号或版本不匹配时返回 `STALE`。只有持有 `ORCHESTRATE` 的 `ORCHESTRATOR_BREAK` 会释放活动租约；声明的 `BREAK_EXPIRED` 和未知标志在当前实现中都不会改变处理。

代码 23: Nexus 结果文件的顺序写入与非原子发布

c

```

1518 static int nexus_artifact_store(
1519     struct agent_nexus_artifact_header *stored, const void *payload,
1520     unsigned int payload_size)
1521 {
1522     struct agent_nexus_artifact_header digest_header;
1523     struct agent_nexus_artifact_header pending_header;
1524     struct agent_file_edit_state store_lock;
1525     char path[AGENT_NEXUS_ARTIFACT_PATH_SIZE];
1526     unsigned int invalid_magic = 0;
1527     int lock_active = 0;
1528     int created = 0;
1529     int existing;
1530     int fd = -1;
1531
1532     if (payload == 0 || payload_size == 0 ||
1533         payload_size > AGENT_NEXUS_ARTIFACT_MAX ||
1534         agent_nexus_artifact_path(stored->handle, path) < 0)
1535         return -1;
1536     agent_nexus_sha256(payload, payload_size, stored->payload_sha256);
1537     memset(stored->manifest_sha256, 0, sizeof(stored->manifest_sha256));
1538     digest_header = *stored;
1539     agent_nexus_sha256(&digest_header, sizeof(digest_header),
1540                     stored->manifest_sha256);
1541
1542     /*
1543      * uCore has no atomic link/rename commit. A workflow-scoped edit lease
1544      * serializes every publisher. The final header is written last, so a
1545      * crash leaves an invalid, publish-once tombstone rather than a readable

```

```

1546     * partial artifact.  AGENT_NEXUS_ARTIFACT_PUBLISH_IS_ATOMIC documents
1547     * this deliberately non-atomic MVP boundary.
1548     */
1549     fd = open(NEXUS_ARTIFACT_STORE_LOCK, O_CREATE | O_WRONLY);
1550     if (fd < 0)
1551         return -1;
1552     existing = close(fd);
1553     fd = -1;
1554     if (existing < 0)
1555         return -1;
1556     memset(&store_lock, 0, sizeof(store_lock));
1557     if (agent_file_edit_begin(NEXUS_ARTIFACT_STORE_LOCK, 0,
1558                             AGENT_FILE_EDIT_MAX_TTL, &store_lock) !=
1559         AGENT_STATUS_OK)
1560         return -1;
1561     lock_active = 1;
1562     existing = open(path, O_RDONLY);
1563     if (existing >= 0) {
1564         (void)close(existing);
1565         goto fail;
1566     }
1567     fd = open(path, O_CREATE | O_WRONLY | O_TRUNC);
1568     if (fd < 0)
1569         goto fail;
1570     created = 1;
1571     pending_header = *stored;
1572     pending_header.magic = 0;
1573     if (nexus_write_all(fd, &pending_header, sizeof(pending_header)) < 0 ||
1574         nexus_write_all(fd, payload, payload_size) < 0 || fsync(fd) < 0)
1575         goto fail;
1576     existing = close(fd);
1577     fd = -1;
1578     if (existing < 0)
1579         goto fail;
1580     fd = open(path, O_WRONLY);
1581     if (fd < 0 || nexus_write_all(fd, stored, sizeof(*stored)) < 0 ||
1582         fsync(fd) < 0)
1583         goto fail;
1584     existing = close(fd);
1585     fd = -1;
1586     if (existing < 0)
1587         goto fail;
1588     if (agent_file_edit_commit(store_lock.lease_id,
1589                             store_lock.base_version, &store_lock) !=
1590         AGENT_STATUS_OK)
1591         goto fail;
1592     lock_active = 0;
1593     return 0;
1594
1595 fail:
1596     if (fd >= 0)
1597         (void)close(fd);
1598     if (created && lock_active) {
1599         fd = open(path, O_WRONLY);
1600         if (fd >= 0) {
1601             (void)nexus_write_all(fd, &invalid_magic,
1602                                 sizeof(invalid_magic));

```

```
1603         (void)fsync(fd);
1604         (void)close(fd);
1605     }
1606 }
1607 if (lock_active)
1608     (void)agent_file_edit_abort(store_lock.lease_id);
1609 return -1;
1610 }
```

4. 项目测试

测试分为五个层次：结构约定检查、交叉编译、RISC-V 客户机场景、宿主机运行时和一次性性能实验。功能结果来自真实系统调用输出和协议摘要；性能数据保留逐样本来源，因此配对比较、分布图和统计表都能追溯到对应的启动记录。

4.1. 功能与性能测试

4.1.1. 测试组织

AgentOS-uCore 的测试分为五个层次：结构约定检查、交叉编译、RV64 客户机场景、宿主机运行时和综合性能。结构约定检查 UAPI 的尺寸、偏移和模块依赖；交叉编译把真实内核和用户目标文件链接为 RISC-V 镜像；客户机场景通过系统调用观察智能体身份、上下文、VFS、IPC、资源和调度状态；宿主机测试覆盖帧编码、服务提供方、N1、任务产物和校验程序。

模块	主要程序	关键场景
身份与上下文	agenttrust_ucore, agentscope_ucore, agentfinal_ucore	创建、派生、执行和退出；代次重用；上下文快照、分支与淘汰
结构化工具	agenttoolabi_ucore, agentcontract_ucore, agenttask_ucore	带类型信息的参数规则；V3 DAG；重试、取消；SQ/CQ 背压与重新同步
数据来源与工作流程屏障	agentfinal_ucore、屏障 与来源固定测试	来源传播、副作用前检查、关键记录环、质询回执
实时查询文件系统	agentfs_ucore, agentbench_ucore	实例号、遍历与索引一致性、ENTER、UPDATE、LEAVE、重新同步
智能体循环与 EEVDF	agentloop_ucore, agentsched_ucore, agent_eevdf_ucore	原子等待、唤醒、超时、心跳、线程放大、Jain 公平性
控制台与 Nexus	labdemo_ucore, agentnexus_ucore	多模块恢复、3 轮交互、4 个业务角色、重新规划、审批拒绝、会话关闭

表 4.1: 六项核心机制的测试场景与程序

当前版本通过 545 项智能体模块与 ABI 结构约定检查、114 项 Nexus 结构约定检查以及 93 个宿主机测试程序。严格的 Nexus 固定回放在一次全新 QEMU 启动中完成 11 个请求摘要、3 轮交互、4 个业务角色、一次失败后的重新规划、一次审批拒绝和完整关闭。

4.1.2. 一次性增强实验

为保留绘图所需的逐样本数据，项目在提交 2b14fb1f74b9bd093e6de939a16554620835699e 上完成了一次性增强实验。实验使用 30 次全新 QEMU 启动，保存 33 份原始串口输出、19 个 CSV 数据表和 7,498 条结构化记录。每条记录都包含来源文件、来源 SHA-256 和原文序号。

实验族	设计	独立单位	样本量
遍历与索引查询	16 个 AB/BA 均衡配对启动	启动记录	16 组核心路径与端到端路径配对
文件目录网格	3 × 4 参数网格，每格 15 个内部配对	源启动记录	180 组查询配对
批量、单项和 SQ-CQ	4 次启动，8 组轮换顺序	启动记录/区段	96 条 16 次操作序列
工作流 EEVDF	6 次启动，并发度为 1–4	启动记录/时间窗和探针	504 条精确唤醒探针

表 4.2: 一次性实验组成

同一次启动中的内部配对用于观察稳定工作量下的重复波动；跨启动汇总时，以源启动记录作为独立单位。所有图表都直接由 CSV 数据表生成，原始顺序、目标对象、工作量参数和警告字段均保留在数据中。

4.1.3. 任务通道传输延迟

任务通道实验让紧凑批量调用、单项 V3 调用和 SQ/CQ 分别完成 16 次等价 ECHO，并用轮换顺序平衡三条路径的先后次序。每条操作序列记录完整的墙钟时间，同时核对工具、状态、上下文序号、执行记录票号和结果指纹的一致性。

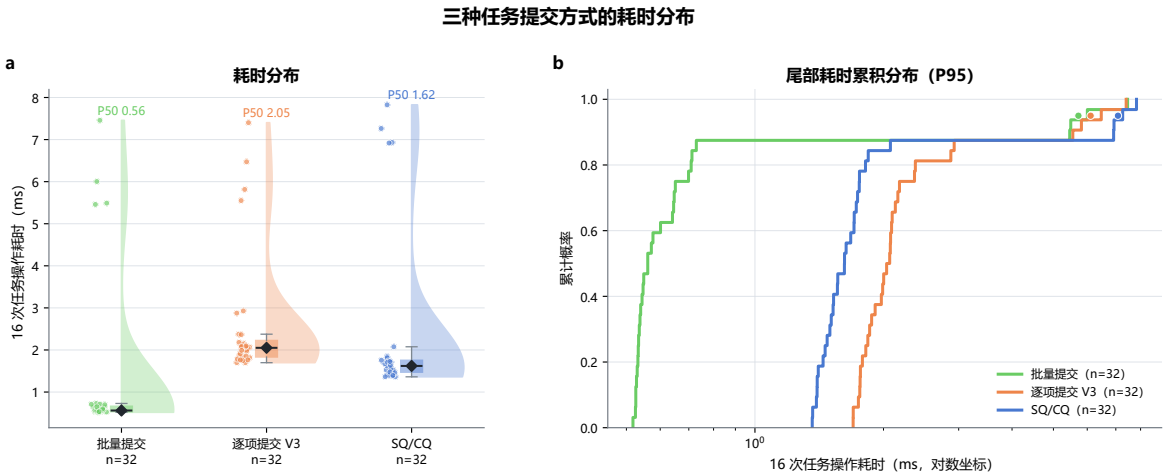


图 4.1: 三种任务提交方式的耗时分布

紧凑批量调用、单项 V3 调用和 SQ/CQ 的操作序列中位数分别为 561 微秒、2051 微秒和 1620.5 微秒。紧凑批量调用在一次系统调用内顺序执行 16 项操作，入口开销最低；V3 每次都绑定执行约定、节点和尝试号，并执行完整执行约定检查；SQ/CQ 增加环形队列提交、完成队列和背压处理，因此延迟介于两条同步路径之间。

4.1.4. 内核 I/O 与工作量

性能实验同时记录文件读写、系统调用、扫描记录数、任务完成次数和工作流服务次数。图4.2分为上下两个面板：上半图把工作流核心路径的 bytes_read 和端到端路径的全局 I/O 计数，分别除以核心时间窗内的系统调用次数；下半图按已完成操作数归一化智能体任务的传输成本。

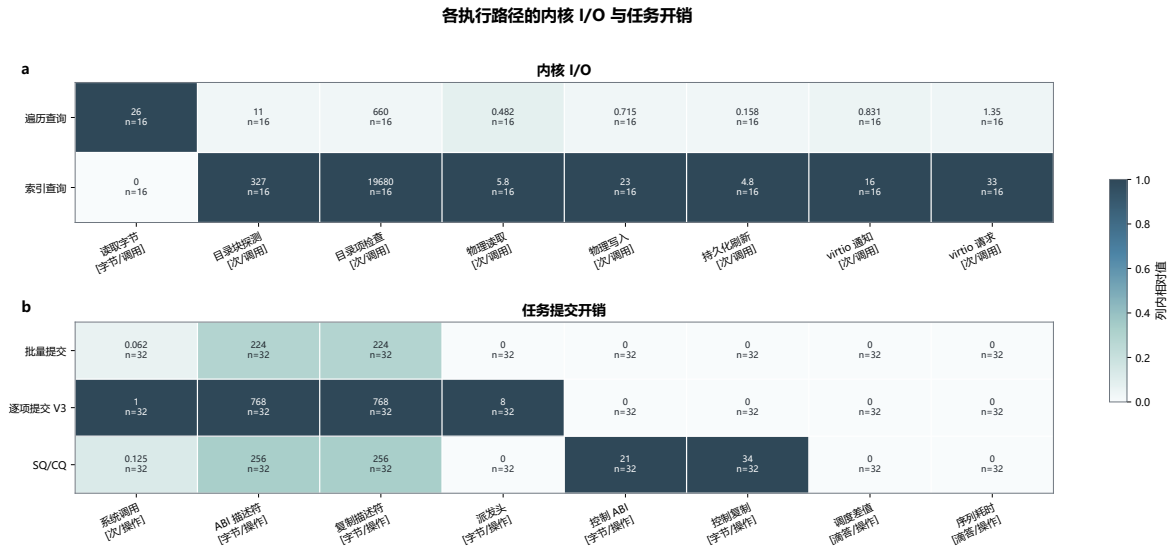


图 4.2: 各执行路径的内核 I/O 与任务开销

每一行都使用独立的量纲和色标，用于比较同一指标在不同路径下的相对变化。

4.1.5. 可重复性

一次性数据集的清单记录源码提交、工具版本、工作量计划、客户机和构建哈希，并汇总原始数据、CSV 数据表和图像文件哈希。清单中的 75 项均通过哈希校验。数据用于离线校验和绘图，日常回归继续运行常规客户机负载。

4.2. 测试路径与结果

测试不只比对用户态输出文本，还通过真实 RV64 客户机调用观察内核对象的状态、返回码、上下文序号、代次和运行统计。功能场景、机制探针和性能测量共用同一构建产物，因此表 4.1 中的项目都能追溯到实际执行路径。

4.2.1. 客户机原生工作负载与一次性测量

labdemo_ucore 的原生工作负载在客户机中创建和使用 AgentOS 对象，收集各机制路径上的计数和延迟。它与兼容工作负载共用报告结构，因此比较不会把宿主机预处理、日志格式或测试框架差异误认为内核性能差异。

代码 24: 客户机原生 AgentOS 工作负载的执行入口

c

```
745 static void run_native_workload(struct labdemo_workload_metrics *metrics)
746 {
747     static struct agent_file_query query;
748     static struct agent_file_query_result result;
749     static struct agent_file_meta target;
750     struct labdemo_lane_measurement_state *state = &native_measurement;
751     static char name[6];
752
753     memset(metrics, 0, sizeof(*metrics));
754     demo_quiescence_fence("native", "E2E_START", 1, &state->e2e_start);
```

```

755 metrics->end_to_end_started_us = demo_now_us();
756 seed_native_workload(&target);
757 memset(&query, 0, sizeof(query));
758 query.flags = AGENT_FILE_QUERY_USE_INDEX;
759 query.max_hits = AGENT_FILE_QUERY_MAX_HITS;
760 strcpy(query.project, DEMO_PROJECT);
761 strcpy(query.run_id, DEMO_BENCH_RUN);
762 strcpy(query.status, "failed");
763 demo_quiescence_fence("native", "CORE_START", 2, &state->core_start);
764 metrics->started_us = demo_now_us();
765 DEMO_DIAG_BEGIN();
766 check(agent_file_query(&query, &result) == 1,
767       "native failed-stage query");
768 DEMO_DIAG_END("native", "failed_index_query");
769 metrics->records_examined += result.scanned_records;
770 check(result.returned == 1 && result.used_index == 1 &&
771       strcmp(result.hits[0].stage, "align") == 0 &&
772       strcmp(result.hits[0].status, "failed") == 0,
773       "native failed-stage result");
774 metrics->discovered_us = demo_now_us();
775 demo_corpus_name(name, 'n', DEMO_BENCH_TARGET);
776 DEMO_DIAG_BEGIN();
777 {
778     int fd = open(name, O_WRONLY | O_TRUNC);
779
780     check(fd >= 0, "native recovery open");
781     check(write(fd, DEMO_BENCH_RECOVERED_BODY,
782               strlen(DEMO_BENCH_RECOVERED_BODY)) ==
783           (ssize_t)strlen(DEMO_BENCH_RECOVERED_BODY),
784           "native recovery write");
785     target.update_mask = AGENT_FILE_META_UPDATE_STATUS |
786                       AGENT_FILE_META_UPDATE_SUMMARY;
787     strcpy(target.status, "recovered");
788     strcpy(target.summary, "memory_limit recovered");
789     check(agent_file_meta_set(&target) == 0,
790           "native recovery metadata stage");
791     check(demo_quiescence_fsync(fd) == 0,
792           "native recovery primary ack");
793     check(close(fd) == 0, "native recovery close");
794 }
795 DEMO_DIAG_END("native", "grouped_recovery_primary_ack");
796 metrics->committed_us = demo_now_us();
797 strcpy(query.status, "recovered");
798 memset(&result, 0, sizeof(result));
799 DEMO_DIAG_BEGIN();
800 check(agent_file_query(&query, &result) == 1,
801       "native recovered-stage query");
802 DEMO_DIAG_END("native", "recovered_index_query");
803 metrics->records_examined += result.scanned_records;
804 check(result.returned == 1 && result.used_index == 1 &&
805       strcmp(result.hits[0].stage, "align") == 0 &&
806       strcmp(result.hits[0].status, "recovered") == 0,
807       "native recovered-stage result");
808 metrics->result_items = 1;
809 metrics->outcome_hash = demo_outcome_hash(result.hits[0].stage,
810                                           result.hits[0].status);
811 metrics->finished_us = demo_now_us();

```

```

812     take_performance_snapshot(&state->core_ack);
813     metrics->workload_syscalls = performance_delta(
814         state->core_start.performance.snapshot.observer_workload_syscalls,
815         state->core_ack.snapshot.observer_workload_syscalls);
816     check(metrics->workload_syscalls != 0, "native workload syscall receipt");
817     demo_quiescence_fence("native", "ACK_SETTLED", 3,
818         &state->ack_settled);
819     DEMO_DIAG_BEGIN();
820     for (int i = 0; i < LABDEMO_CORPUS_SIZE; i++) {
821         demo_corpus_name(name, 'n', i);
822         check(unlink(name) == 0, "native corpus cleanup");
823     }
824     DEMO_DIAG_END("native", "cleanup_unlink");
825     demo_quiescence_fence("native", "E2E_END", 4, &state->e2e_end);
826     metrics->end_to_end_finished_us = demo_now_us();
827     print_workload_lane("native", metrics);
828     print_mechanism_delta("native", "core", &state->core_start.performance,
829         &state->core_ack);
830     print_mechanism_delta("native", "end_to_end",
831         &state->e2e_start.performance,
832         &state->e2e_end.performance);
833 }
834
835 static void make_op(struct agent_op *op, int tool, uint64 id, uint64 arg0,
836     const char *payload)
837 {
838     memset(op, 0, sizeof(*op));
839     op->version = AGENT_CALL_VERSION;
840     op->tool_id = tool;
841     op->request_id = id;
842     op->arg0 = arg0;
843     if (payload)
844         strcpy(op->payload, payload);
845 }

```

一次性实验图表由当前测量目录中的原始数据生成。核心路径与端到端窗口的对比见第3.6章图3.5；该图保留机制路径上的时间和资源观测值。所有数值对应当前 QEMU/uCore 配置、给定工作负载和该轮运行，跨硬件平台的吞吐量仍需另行测量。

4.2.2. 工具、执行约定与任务通道测试

Nexus 运行时发起工具调用时，会把任务文本编码为固定结构，并通过 `send_message` 进入带类型信息的工具路径。该路径先完成任务结构校验、参数编码和回包状态检查，因此测试能区分用户态编码失败、内核参数规则拒绝和工具副作用失败。

代码 25: Nexus 任务编码与 `send_message` 调用

c

```

992 int agent_nexus_task_send(int target_pid, unsigned long long task_id,
993     const struct agent_nexus_task *task,
994     struct agent_response_v2 *response)
995 {
996     struct agent_nexus_tool_argument arguments[2];
997     char message[AGENT_EVENT_PAYLOAD_SIZE];
998 }

```

```

999     if (target_pid <= 0 || task_id == 0 || response == 0 ||
1000         agent_nexus_task_encode(task, message) < 0)
1001         return -1;
1002     memset(arguments, 0, sizeof(arguments));
1003     strcpy(arguments[0].key, "target_pid");
1004     arguments[0].type = AGENT_PARAM_UINT64;
1005     arguments[0].number = (unsigned long long)target_pid;
1006     strcpy(arguments[1].key, "message");
1007     arguments[1].type = AGENT_PARAM_STRING;
1008     strcpy(arguments[1].text, message);
1009     if (agent_nexus_tool_call("send_message", task_id, arguments, 2,
1010         response) < 0)
1011         return -1;
1012     return response->status;
1013 }

```

执行约定测试覆盖过期生命周期、过期执行约定、未知节点、工具不匹配、截止时间和依赖完成状态等分支。副作用前检查产生的完成记录关联执行记录票号和输出来源；对于直接调用或 V3 调用的拒绝，如果无法建立关键记录，调用会返回 NO_SPACE/RETRY，测试同样覆盖这条记录失败语义。因此，拒绝或取消不是孤立的错误码，而是能够与上下文、执行记录或明确的记录失败状态关联的结果。

代码 26：执行约定完成时的执行记录与完成状态不变量

c

```

1834 void
1835 agent_execution_contract_complete(struct agent_execution_claim *claim,
1836     const struct agent_result *result,
1837     const struct agent_execution_outcome *outcome)
1838 {
1839     struct agent_execution_contract_record *record;
1840     struct agent_execution_node_internal *node;
1841     struct agent_execution_node_runtime *runtime;
1842     struct agent_execution_completion *cached;
1843     struct agent_execution_producer_metadata *producer;
1844     int cache_index;
1845     int terminal_failure;
1846     int enabled;
1847
1848     if (claim == 0 || result == 0 || !claim->active || claim->legacy ||
1849         claim->slot < 0 ||
1850         claim->slot >= WORKFLOW_LIFECYCLE_CAP ||
1851         claim->node_id >= AGENT_EXECUTION_CONTRACT_MAX_NODES)
1852         return;
1853     enabled = intr_save();
1854     record = &agent_execution_contracts[claim->slot];
1855     if (!agent_execution_record_matches(record, claim->lifecycle) ||
1856         record->generation != claim->contract_generation ||
1857         claim->node_id >= record->node_count) {
1858         intr_restore(enabled);
1859         return;
1860     }
1861     runtime = &record->runtime[claim->node_id];
1862     node = &record->nodes[claim->node_id];
1863     if (runtime->state != AGENT_EXECUTION_NODE_RUNNING ||
1864         runtime->attempt_id != claim->attempt_id ||

```

```

1865     !agent_execution_digest_equal(runtime->request_digest,
1866                                   claim->request_digest)) {
1867         intr_restore(enabled);
1868         return;
1869     }
1870     if ((runtime->flags &
1871          AGENT_EXECUTION_RUNTIME_F_CANCEL_REQUESTED) != 0 &&
1872         result->status != AGENT_STATUS_CANCELLED)
1873         panic("execution contract cancel winner");
1874     if ((runtime->flags &
1875          AGENT_EXECUTION_RUNTIME_F_TIMEOUT_REQUESTED) != 0 &&
1876         result->status != AGENT_STATUS_TIMEOUT)
1877         panic("execution contract timeout winner");
1878     if (result->sequence == 0 || outcome == 0 ||
1879         outcome->evidence_ticket == 0 ||
1880         outcome->output_provenance_labels == 0)
1881         panic("execution contract terminal without evidence");
1882     runtime->sequence = result->sequence;
1883     runtime->status = result->status;
1884     runtime->decision_reason = claim->decision_reason;
1885     record->running_mask &= ~(1ULL << claim->node_id);
1886     if (record->running_count == 0)
1887         panic("execution contract running underflow");
1888     record->running_count--;
1889     producer = &agent_execution_producers[claim->slot][claim->node_id];
1890     if (claim->producer_control_id == 0 || claim->producer_pid <= 0)
1891         panic("execution contract producer identity");
1892     producer->control_id = claim->producer_control_id;
1893     producer->pid = claim->producer_pid;
1894     producer->context_sequence = result->sequence;
1895     producer->provenance_labels = outcome->output_provenance_labels;
1896     producer->valid = 1;
1897     if (result->status == AGENT_STATUS_OK) {
1898         runtime->state = AGENT_EXECUTION_NODE_SUCCEEDED;
1899         record->completed_mask |= 1ULL << claim->node_id;
1900         record->failed_mask &= ~(1ULL << claim->node_id);
1901     } else {
1902         runtime->state = result->status == AGENT_STATUS_CANCELLED ?
1903             AGENT_EXECUTION_NODE_CANCELLED :
1904             AGENT_EXECUTION_NODE_FAILED;
1905         record->completed_mask &= ~(1ULL << claim->node_id);
1906         terminal_failure = claim->retry_forbidden ||
1907             claim->attempt_id >= node->max_attempts ||
1908             !agent_execution_retry_allowed(node, result->status);
1909         if (terminal_failure) {
1910             record->failed_mask |= 1ULL << claim->node_id;

```

任务通道在提交后维护已接纳和运行状态、生命周期引用以及回调引用；硬截止时间触发过期，完成或拒绝进入统一的完成队列。该状态机的测试同时验证“不丢失生命周期引用”和“不在过期后伪装完成”两项性质。

代码 27：任务提交后的回调、截止时间与完成分支

c

```

1880     request->sqe = sqe;
1881     request->accepted_tick = agent_task_now();

```

```

1882     request->expected_output_type = validation.output_artifact_type;
1883     request->provenance_labels = validation.output_provenance_labels;
1884     request->state = AGENT_TASK_REQUEST_ACCEPTED;
1885     request->flags = AGENT_TASK_REQUEST_F_LIFECYCLE_HELD;
1886     if (private->header.callback_refs == 0)
1887         panic("Task Channel submit transfer");
1888     agent_task_callback_put_locked(state);
1889     if (agent_task_request_input_acquire_locked(
1890         private, state->resource_private, request) < 0) {
1891         request->flags &= ~AGENT_TASK_REQUEST_F_LIFECYCLE_HELD;
1892         workflow_lifecycle_operation_leave(lifecycle);
1893         memset(request, 0, sizeof(*request));
1894         status = agent_task_channel_protocol_fault_locked(state);
1895         intr_restore(enabled);
1896         return status;
1897     }
1898     request->state = AGENT_TASK_REQUEST_RUNNING;
1899     private->header.sq_head++;
1900     private->header.submitted++;
1901     private->header.last_accepted_request_id = sqe.request_id;
1902     *consumed = 1;
1903     private->header.live_requests++;
1904     agent_task_callback_get_locked(state);
1905     agent_task_channel_publish_locked(state);
1906     intr_restore(enabled);
1907     hook_status = ops->submit(
1908         p, &sqe, &input_view, &validation, &completion);
1909     enabled = intr_save();
1910     state = agent_task_channel_find_locked(p);
1911     if (!agent_task_channel_owner_valid_locked(state, p) ||
1912         state->ops != ops || state->private->header.callback_refs == 0)
1913         panic("Task Channel submit callback");
1914     agent_task_callback_put_locked(state);
1915     intr_restore(enabled);
1916     agent_task_reclaim_deferred(p, ops);
1917     if (hook_status == AGENT_TASK_HOOK_PENDING &&
1918         (sqe.flags & AGENT_TASK_SQE_F_HARD_DEADLINE) != 0 &&
1919         agent_task_now() >= sqe.deadline_tick) {
1920         status = agent_task_channel_expire(p, agent_task_now(), ops);
1921         return agent_task_consumed_after_expire(status);
1922     }
1923     if (hook_status == AGENT_TASK_HOOK_COMPLETE ||
1924         hook_status == AGENT_TASK_HOOK_DENIED) {
1925         status = agent_task_channel_complete(
1926             p, sqe.ring_generation, sqe.request_id,
1927             sqe.slot_generation, &completion, ops);
1928         if (status == AGENT_TASK_CHANNEL_OK)
1929             return 1;
1930         return status;
1931     }
1932     if (hook_status != AGENT_TASK_HOOK_PENDING)
1933         return AGENT_TASK_CHANNEL_EVIDENCE;
1934     return 1;

```

4.2.3. Nexus 会话与宿主机中继程序

Nexus 使用带序号的帧，把客户机的会话、工具事件、审批和运行指标接入宿主机中继程序。用户态解码器会校验会话、序号、类别和长度，再由中继程序完成服务提供方适配和本地端点转发。网络和模型协议留在宿主机，智能体身份、工具权限和任务状态仍由客户机和内核验证。

代码 28: Nexus V2 帧的编码入口

c

```

903 static int live_frame_encode(const char *session, uint64 sequence,
904                             const char *kind, const char *payload,
905                             char *output, uint capacity)
906 {
907     struct live_builder builder;
908     unsigned char digest[LIVE_SHA_SIZE];
909     char digest_hex[LIVE_SHA_HEX_SIZE + 1];
910     uint payload_length = strlen(payload);
911     uint encoded_length;
912
913     if (!live_session_valid(session) || sequence == 0 ||
914         !live_kind_valid_v2(kind) ||
915         payload_length == 0 ||
916         payload_length > LIVE_MAX_JSON ||
917         !live_utf8_valid((const unsigned char *)payload, payload_length))
918         return -1;
919     live_sha256(payload, payload_length, digest);
920     live_digest_hex(digest, digest_hex);
921     encoded_length = live_base64_encode(
922         (const unsigned char *)payload, payload_length,
923         live_base64_buffer, sizeof(live_base64_buffer));
924     if (encoded_length == 0)
925         return -1;
926     live_builder_init(&builder, output, capacity);
927     live_builder_text(&builder, LIVE_PREFIX_V2);
928     live_builder_text(&builder, session);
929     live_builder_char(&builder, ' ');
930     live_builder_u64(&builder, sequence);
931     live_builder_char(&builder, ' ');
932     live_builder_text(&builder, kind);
933     live_builder_char(&builder, ' ');
934     live_builder_u64(&builder, payload_length);
935     live_builder_char(&builder, ' ');
936     live_builder_text(&builder, digest_hex);
937     live_builder_char(&builder, ' ');
938     live_builder_text(&builder, live_base64_buffer);
939     live_builder_char(&builder, '\n');
940     return builder.ok && builder.length <= LIVE_MAX_FRAME ?
941         (int)builder.length : -1;
942 }

```

代码 29: 宿主机中继程序的串口写入与本地消息分发

Python

```

2154 def _reader(self, stream, source: str) -> None:
2155     def run() -> None:

```



```

2156         while True:
2157             try:
2158                 chunk = stream.read(4096)
2159             except OSError:
2160                 chunk = b""
2161             if not chunk:
2162                 self.events.put(("eof", source.encode("ascii")))
2163             return
2164             self.events.put((source, chunk))
2165
2166         threading.Thread(target=run, name=f"agentos-{source}", daemon=True).start()
2167
2168     def _write_serial(self, line: bytes) -> None:
2169         relay._write_process_before_deadline(
2170             self.process,
2171             line,
2172             deadline_monotonic=time.monotonic() + SERIAL_WRITE_TIMEOUT_SECONDS,
2173         )
2174
2175     def _drain_local(self) -> None:
2176         # Bound each pass so local traffic cannot defer provider polling or
2177         # Guest serial processing. The role-aware queue also prevents a ping
2178         # flood from placing a controller command behind every observer item.
2179         for _ in range(MAX_LOCAL_DRAIN):
2180             try:
2181                 peer, message = self.endpoints.inbound.get_nowait()
2182             except queue.Empty:
2183                 return
2184             try:
2185                 # A peer may disconnect after its reader queued commands.
2186                 # Identity and liveness are checked at execution time so a
2187                 # replacement controller never inherits the old queue.
2188                 if not self.endpoints.is_current(peer):
2189                     continue
2190             try:
2191                 self._handle_local(peer, message)
2192             except relay.RelayError as error:
2193                 peer.send(
2194                     {
2195                         "type": "error",
2196                         "code": error.code,
2197                         "message": error.public_message,
2198                     }
2199                 )
2200             finally:
2201                 consumed = getattr(peer, "inbound_consumed", None)
2202                 if consumed is not None:
2203                     consumed()
2204
2205     def _handle_local(self, peer: _Peer, message: Mapping[str, object]) -> None:
2206         message_type = message.get("type")
2207         if peer.role == "observer":
2208             if message_type == "ping" and set(message) == {"type"}:
2209                 peer.send({"type": "pong"})
2210             return
2211         raise relay.WireProtocolError("ROLE_DENIED", "observer is read-only")
2212         if message_type == "user_message":

```

```
2213         if set(message) != {"type", "content"}:
2214             raise relay.WireProtocolError("BAD_LOCAL_MESSAGE", "user message is malformed")
2215         self.session.submit_user(message.get("content"))
2216         elif message_type == "command":
2217             if set(message) != {"type", "command"}:
2218                 raise relay.WireProtocolError("BAD_LOCAL_MESSAGE", "control message is
malformed")
2219             command = message.get("command")
2220             self.session.request_control(command)
2221         elif message_type == "approval":
2222             binding_fields = (
2223                 NEXUS_APPROVAL_BINDING_FIELDS
2224                 if self.guest_profile == "nexus"
2225                 else APPROVAL_BINDING_FIELDS
2226             )
2227             expected = {"type", "decision", *binding_fields}
2228             if set(message) != expected:
2229                 raise relay.WireProtocolError("BAD_LOCAL_MESSAGE", "approval message is
malformed")
2230             self.session.decide_approval(
```

测试运行还检查关闭、取消和过期后的清理过程。宿主机中继程序可以请求停止，客户机把等待和工具事件转换为可观察的完成记录；内核则利用生命周期代次，阻止回收后的延迟消息、任务完成通知或订阅引用落到新的工作流。

4.2.4. 现有测试范围

当前场景覆盖测试环境中的源码和所列客户机负载，检查各机制能否连通以及约束分支能否正确处理。上下文回滚只恢复后续决策可见的上下文路径，不能撤销已完成的外部副作用；任务资源导入仍被拒绝；结果文件发布也不是跨对象的原子提交。真实多核、长时间压力、故障注入和持久化恢复条件下的行为仍需继续测量。

5. 总结与展望

5.1. 阶段性结论

本项目在 RISC-V uCore 中实现了内核中的智能体运行机制。AgentOS-uCore 以现有的进程、页表、VFS、系统调用和调度器为基础，把身份、工具、上下文、文件查询、事件等待、资源和调度纳入同一工作流。六组机制让智能体在发生副作用时都具有明确的身份、能力位、访问范围、代次、数据来源和资源账户。

项目取得了三方面进展。第一，智能体已从应用层标签变为内核能够识别的身份，普通进程和智能体可以在同一个 uCore 中共存。第二，带类型信息的 V2、V3 调用、实时查询、消息等待、资源账户和 EEVDF 覆盖了智能体从调用、等待到取得资源服务的主要过程。第三，控制台和 Nexus 把这些内核机制用于科研恢复场景和四角色常驻协作，使系统既能运行确定性流程，也能支持模型驱动交互。

实验展示了各条路径的性能特征。索引核心路径在 16 个独立配对中均取得更低延迟，配对加速比中位数为 3.118 倍；12 组文件目录网格的中位加速比介于 1.164–2.808 倍；504 条 EEVDF 唤醒探针均在 0–1 个时钟滴答内得到服务。端到端窗口中，索引路径相对遍历路径的中位差值为 +13.452 ms，说明后续优化应继续关注恢复写入、fsync 和复核阶段。

下一阶段将沿三条主线推进：为任务通道增加业务负载后端，使 SQ/CQ 能处理 Nexus 产生的结果文件；为结果文件和工作流屏障增加可选持久化层，支持跨启动周期恢复；将工作流 EEVDF 扩展到 SMP 和真实 RISC-V 硬件，分析多核实体迁移、唤醒和资源记账。这些工作仍以身份、生命周期和工作流资源账户为共同依据，使 AgentOS 保持清晰、紧凑的内核结构。

5.2. 当前技术限制

上下文回滚只恢复后续决策可见的上下文记录和路径状态，**不会撤销已经完成的外部副作用**；幂等副作用仍由 V3 的执行尝试缓存和具体工具协议处理。任务通道已经提供提交、队列、截止时间和完成处理，但当前版本拒绝导入资源。结果文件发布接口仍不是跨对象的原子提交，后续需要进一步协调结果文件、元数据、工作流屏障和持久化事务。

5.3. 后续工作

下一阶段将为任务通道增加业务负载后端，使 SQ/CQ 能处理 Nexus 产生的结果文件；为结果文件和工作流屏障增加可选持久化层，支持跨启动周期恢复；将工作流 EEVDF 扩展到 SMP 和真实 RISC-V 硬件，分析实体迁移、唤醒和资源记账。上述工作仍以身份、生命周期和工作流资源账户作为共同依据。

6. 参考说明

6.1. uCore 与系统基础

AgentOS-uCore 建立在 LearningOS/uCore 教学内核之上，继承 RISC-V 启动、进程、虚拟内存、VFS、异常处理、系统调用和基础调度框架。项目在这些现有对象上增加智能体状态和用户内核接口（UAPI），同时保持原有组件的职责边界。相关基础资料见参考文献中的 uCore 课程材料。

6.2. 协议、工具与运行时

结构化工具和任务通道使用定长结构、版本和大小字段、共享头文件以及客户机与内核布局检查。Nexus 的宿主机中继程序管理本地套接字、TLS、服务提供方适配器和帧转发；客户机保存业务状态、任务产物以及与内核的交互。系统中的外部依赖及许可信息以仓库中的许可证和源码声明为准。

6.3. 测量资料

性能图来自 docs/figures/performance/ 的当前导出文件。一次性增强实验保留原始串口输出、CSV 和逐样本来源信息，用于复核配对延迟、扫描组、任务操作序列和 EEVDF 唤醒探针。图表叙述依据这些保存的测量结果，不以模型推断替代原始记录。

6.4. 文献使用

系统设计参考操作系统、能力安全、结构化接口、事件驱动运行时和公平调度等公开资料。

7. 参考文献

- [1] LearningOS Community. ucore-tutorial-code-2025s. <https://github.com/LearningOS/uCore-Tutorial-Code-2025S>, 2025. AgentOS-uCore 的教学内核基础；访问日期：2026-08-11.
- [2] Linux Kernel Documentation. Cpu accounting controller. <https://docs.kernel.org/6.11/admin-guide/cgroup-v1/cpuacct.html>, 2024. 访问日期：2026-08-11.
- [3] Linux Kernel Project. rstat.c. <https://github.com/torvalds/linux/blob/master/kernel/cgroup/rstat.c>, 2026. 访问日期：2026-08-11.
- [4] Linux Kernel Documentation. Bpf ring buffer. <https://docs.kernel.org/6.6/bpf/ringbuf.html>, 2023. 访问日期：2026-08-11.
- [5] Linux Kernel Documentation. Eevdf scheduler. <https://docs.kernel.org/scheduler/sched-eevdf.html>, 2026. 访问日期：2026-08-11.
- [6] Jens Axboe. Efficient io with io_uring. https://kernel.dk/io_uring.pdf, 2019. 访问日期：2026-08-11.
- [7] Haiku Project. Practical file system design with the be file system. <https://www.haiku-os.org/legacy-docs/practical-file-system-design.pdf>, 1999. 访问日期：2026-08-11.
- [8] Anthropic. How tool use works. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>, 2026. 访问日期：2026-08-11.
- [9] Anthropic. How claude code works: The agentic loop. <https://code.claude.com/docs/en/how-claude-code-works>, 2026. 访问日期：2026-08-11.